Diversity and Distribution course support

Table of contents

1 Motivation **2**

2 I. Species abundance distributions **3**
2.1 Figures . . . 3
2.1.1 Preston diagram . . . 3
2.1.2 Whittaker diagram . . . 5

3 Distributions of abundance **6**
3.1 Broken stick . . . 6
3.2 Geometric . . . 7
3.3 Lognormal . . . 8
3.4 Log-series . . . 9
3.5 Hubbell neutral theory: . . . 12
3.6 Estimate the parameters . . . 12
3.7 Zero-sum multinomial distribution . . . 17

4 Species Accumulation Curves **20**
4.1 BCI data . . . 21
4.2 Accumulation of subplots . . . 22
4.3 Accumulation curve . . . 23
4.4 Diapos 66-68 . . . 24
4.4.1 SAC per area : . . . 24
4.4.2 Spatial structure of the forest : . . . 24
4.4.3 Arrhenius's law . . . 24

5 Canonical lognormal **25**

6 II. Theory of the Measurement of Biodiversity **28**

7 Richness **28**
7.0.1 Jackknife estimators : . . . 28

7.1 Dummy communities . . . 28
7.2 Paracou 6 . . . 29
7.3 Interpolation / extrapolation . . . 32

8 Shannon **33**
8.1 Information function . . . 33
8.2 Paracou 6 . . . 35
8.3 Dummy communities . . . 36

9 Simpson **37**
9.1 Paracou 6 . . . 37

10 Generalized entropy **38**
10.1 Information . . . 38
10.2 Paracou 6 . . . 40
10.3 Dummy communities . . . 43

11 Comparing diversity **43**
11.1 Asymptotic diversity . . . 43
11.2 Standardized diversity . . . 44
11.3 Sample coverage . . . 45

12 Phylogenetic diversity **46**
12.1 Mock data . . . 46
12.2 Paracou 6 . . . 48

13 Functional diversity **50**
13.1 Mock data . . . 50
13.2 Paracou 6 . . . 53
13.3 Intraspecific variability . . . 55

14 Diversity partitioning **58**
14.1 Mock data . . . 58
14.2 Paracou 6 . . . 60
14.2.1 Partitioning . . . 61
14.2.2 Step by step : what the computer does . . . 62
14.2.3 Profiles . . . 63

1 Motivation

This document contains code chunks to help you understanding the course. Don't hesitate to modify them, add your own notes and code.

It comes with a practical you will run at home (`Biodiv-Practical.Rmd`).

2 I. Species abundance distributions

2.1 Figures

2.1.1 Preston diagram

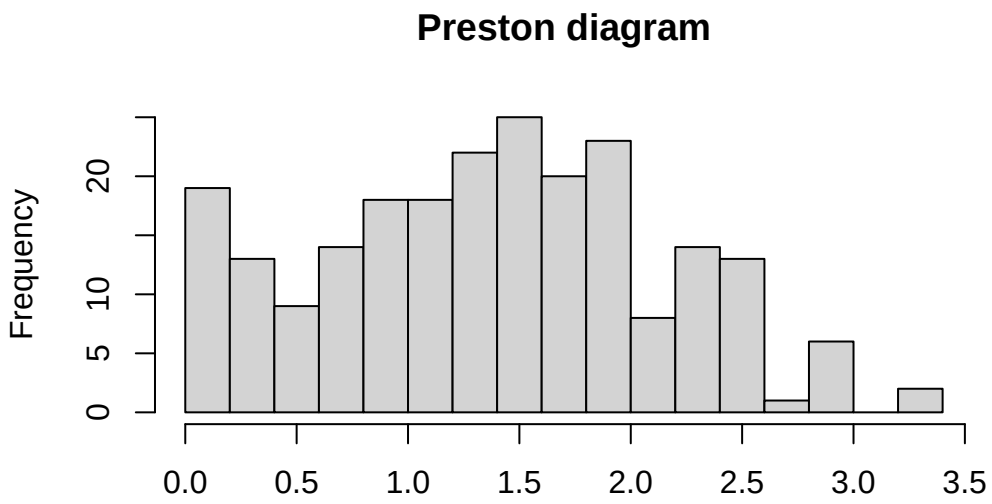
```
library("tidyverse")
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library("vegan")
```

```
Le chargement a nécessité le package : permute
```

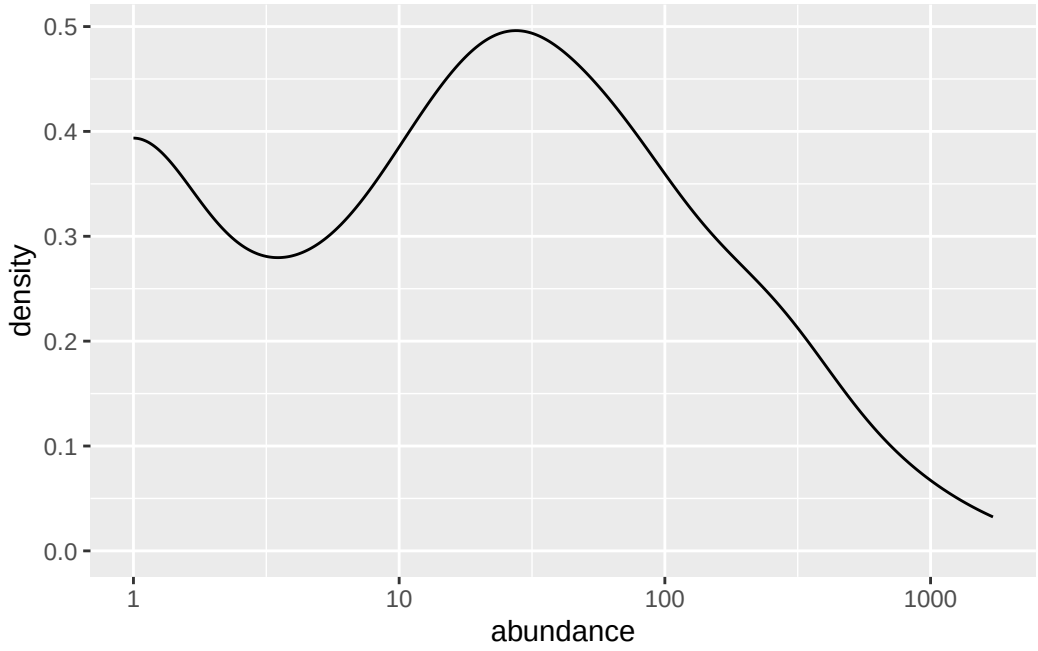
```
data(BCI)
# abondance des espèces dans 50 plots de 1ha
# Sum the 50 1-ha plots
BCI_50ha_abd <- colSums(BCI)
BCI_50ha_abd %>%
  log10() %>%
  hist(breaks = 15, main = "Preston diagram") #breaks=nb de divisions de barre
```



.

Probability densities
ggplot doesn't work with vectors -> we transform it into a tibble

```
BCI_50ha_abd %>%
  tibble(abundance = .) %>%
  ggplot() +
  # Bounded density
  geom_density(aes(x = abundance), bounds = c(0, Inf)) +
  scale_x_log10()
```

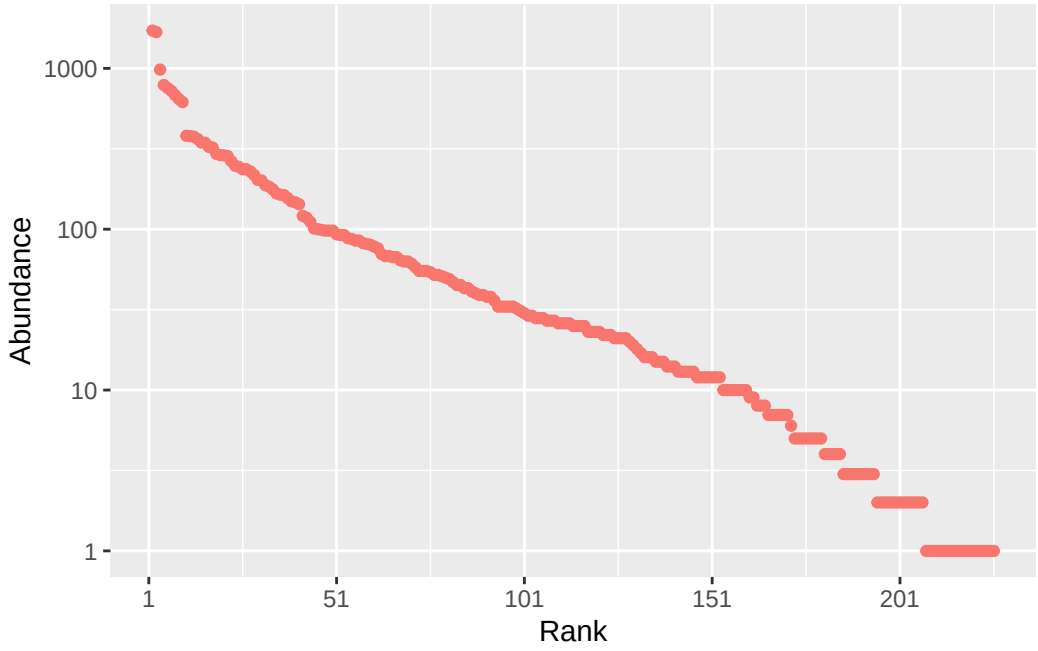


2.1.2 Whittaker diagram

```
library("divent")
```

```
Le chargement a nécessité le package : Rcpp
```

```
BCI_50ha_abd %>%
  # Declare it is an "abundances" object
  as_abundances() %>%
  autoplot()
```



Inductive approach = descriptive
-> on regarde les données et on essaye de sortir des patterns

Deductive approach = mechanistic
-> on crée un modèle et on le compare aux données

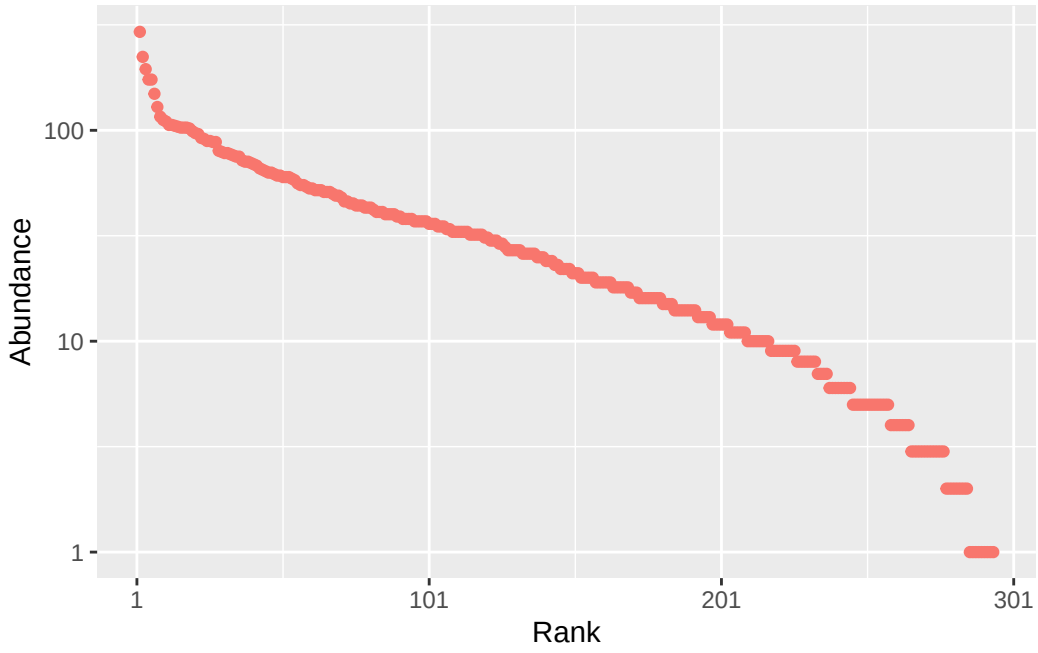
3 Distributions of abundance

3.1 Broken stick

Random distributions are created by `divent::rxxx()`functions.

```
rcommunity(
  # Random drawing of one community
  1,
  # Number of trees
  size = 10000,
  # Number of species
  species_number = 300,
  # Distribution
  distribution = "bstick"
) %>%
```

```
  # Whittaker plot
  autoplot()
```




```
# it's not random : expected values
```

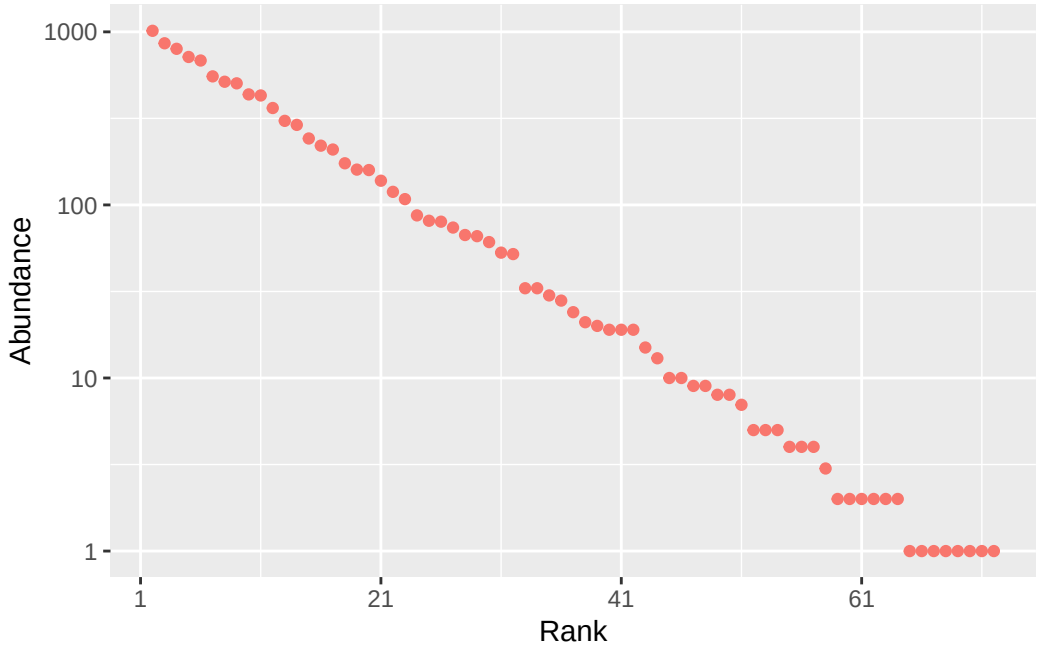
3.2 Geometric

very uneven distribution
applied to ecosystems with a laaaaaaaaaarge number of individuals : example soil microorganisms
-> competitive systems

Random distribution.
Every species takes the same share, one after another.
Example: 1st species takes 10% -> the second will take another 10% of what's left etc

```
rcommunity(
  # One community
  1,
  # Number of trees
  size = 10000,
```

```
  # Number of species
  species_number = 300,
  # Distribution
  distribution = "geom",
  prob_geom = 0.1 #paramètre k : part que prennent les espèces à chaque pas (10%)
) %>%
  # Whittaker plot
  autoplot()
```




```
# nothing is random here
# not strictly aligned here because the abundance is only integer=round values
```

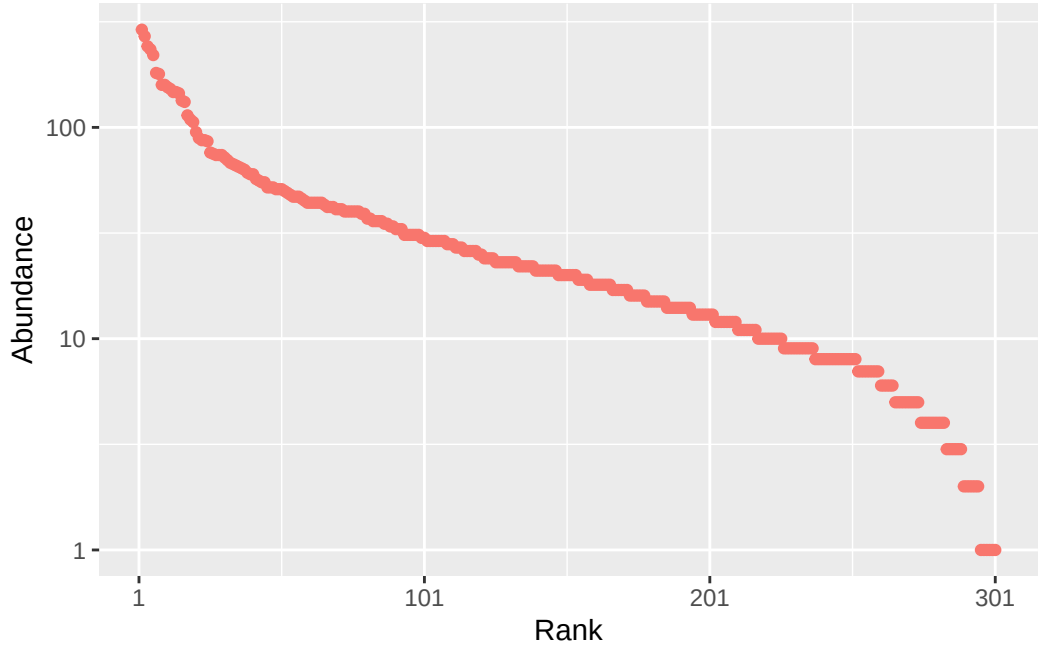
3.3 Lognormal

follows a normal distribution with the central limit theorem
if you sum whatever random variable, it will be normally distributed
-> the logarithm of the product of random variables is a sum -> CLT applies

usually applies to small tropical forests

Random distribution.

```r
rcommunity(
  # One community
  1,
  # Number of trees
  size = 10000,
  # Number of species
  species_number = 300,
  # Distribution
  distribution = "lnorm",
  sd_lnorm = 1
) %>%
  # Whittaker plot
  autoplot()
```



Modify the standard deviation.

3.4 Log-series

on draw n individus d'une espèce -> suit une loi de poisson
the expected number of individuals is a random variable : E(N_s) = lambda_s
but lambda_s is random

negative binomial = number of draws needed to have a certain amount of wins

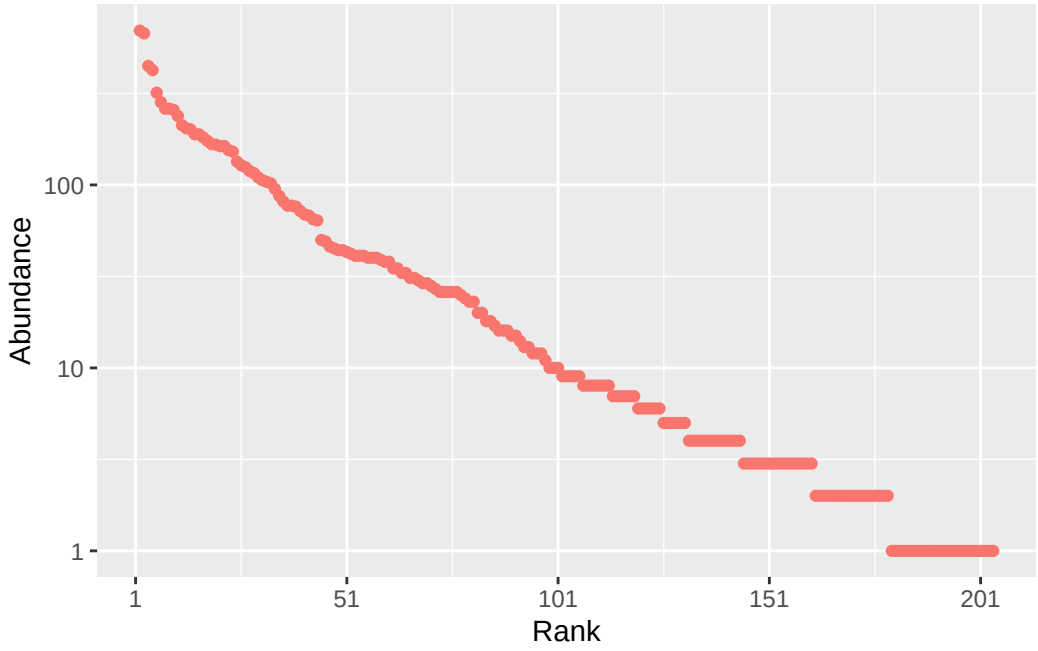
sharp decrease in gamma distribution -> k tend vers 0 donc on approxime k=0
so now that k=0 -> ça revient à une binomiale négative ? jsp
du coup on a E(S_nu)= alpha/nu*x^nu avec x=p/p+1 = close to 1

total number of species : on utilise un développement limité
idem pour nombre total d'individus

alpha is the slope of the relation between the log of the number of species and the log of the number of individuals

Example : mushrooms community
does not work for forests

```
rcommunity(
  # One community
  1,
  # Number of trees
  size = 10000,
  # Number of species is calculated
  # Distribution
  distribution = "lseries",
  fisher_alpha = 40
) %>%
  # Whittaker plot
  autoplot()
```

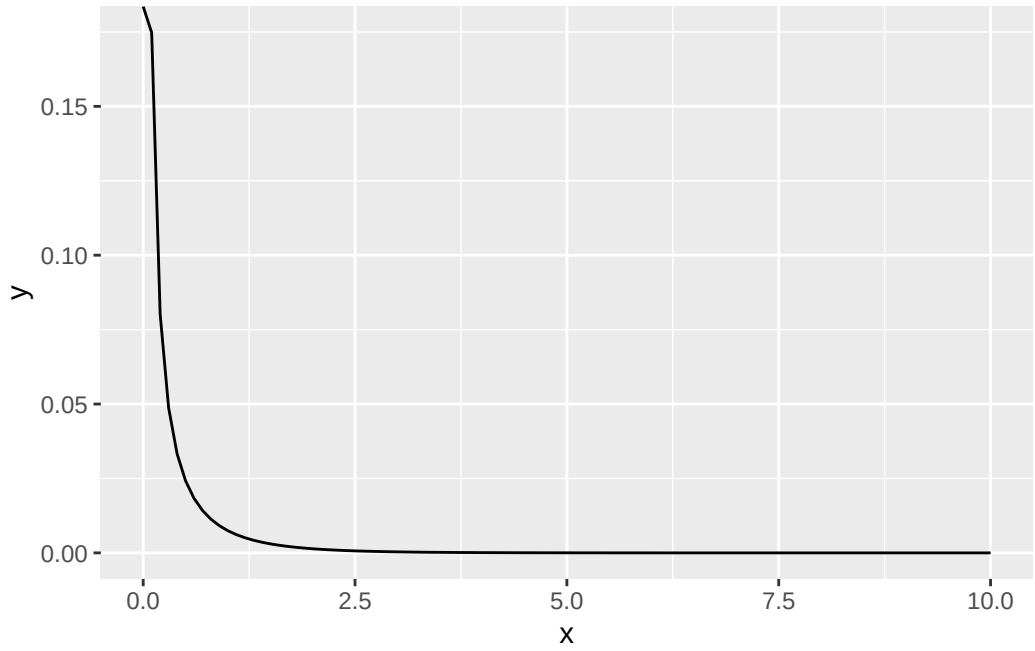


3.4.0.1 The gamma function :

k>1 : forme normale un peu

k<1 : sharp decrease

```
ggplot() + geom_function(fun=dgamma, args=list(shape=0.02), #shape = paramètre k
                         xlim=c(0,10))
```



3.5 Hubbell neutral theory:

migration rate : 10% in tropical rainforests
if no migration -> the community becomes monospecific
This theory is sufficient to explain diversity in tropical forests

number of species in the metacommunity * speciation rate = **TETA** = fundamental number of biodiversity
= how many species appears at each step

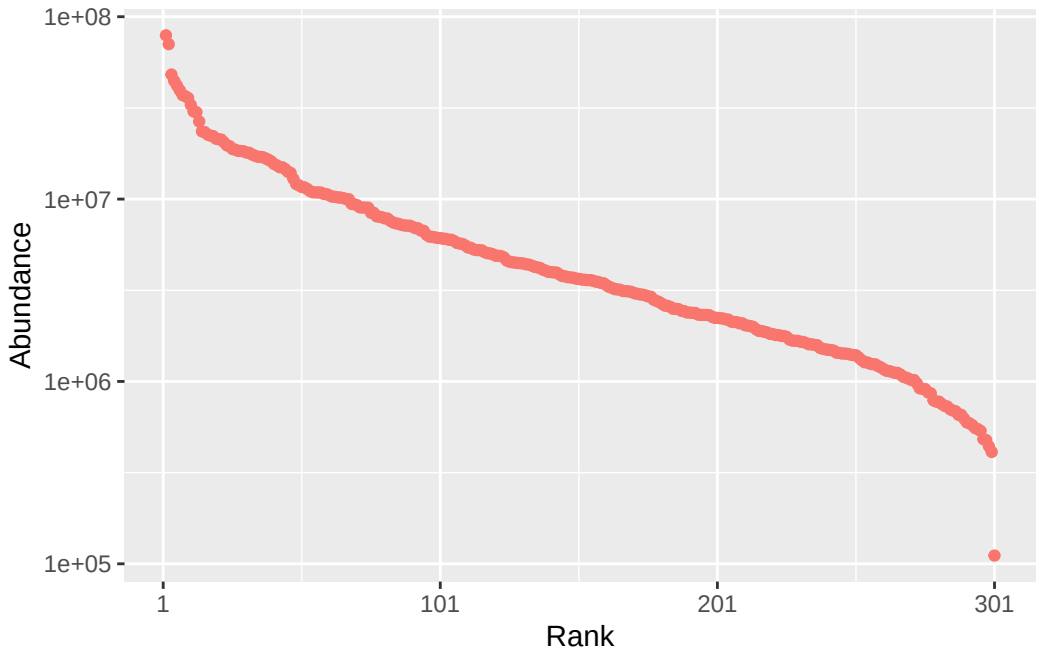
alpha = teta

-> the metacommunity is distributed in log-series
-> All Amazon and all French Guiana is distributed in log-series
but not Paracou (local community) : log-series when area ~ >10,000 ha

3.6 Estimate the parameters

Draw a very large lognormal distribution.

```r
rcommunity(
  # One community
  1,
  # Number of trees = max supported integer value
  size = .Machine$integer.max,
  # Number of species
  species_number = 300,
  # Distribution
  distribution = "lnorm",
  sd_lnorm = 1
) %>%
  # Name it
  mutate(site = "Large log-normal") ->
  comm_large_lnorm
  # Plot it
comm_large_lnorm %>% autoplot()
```

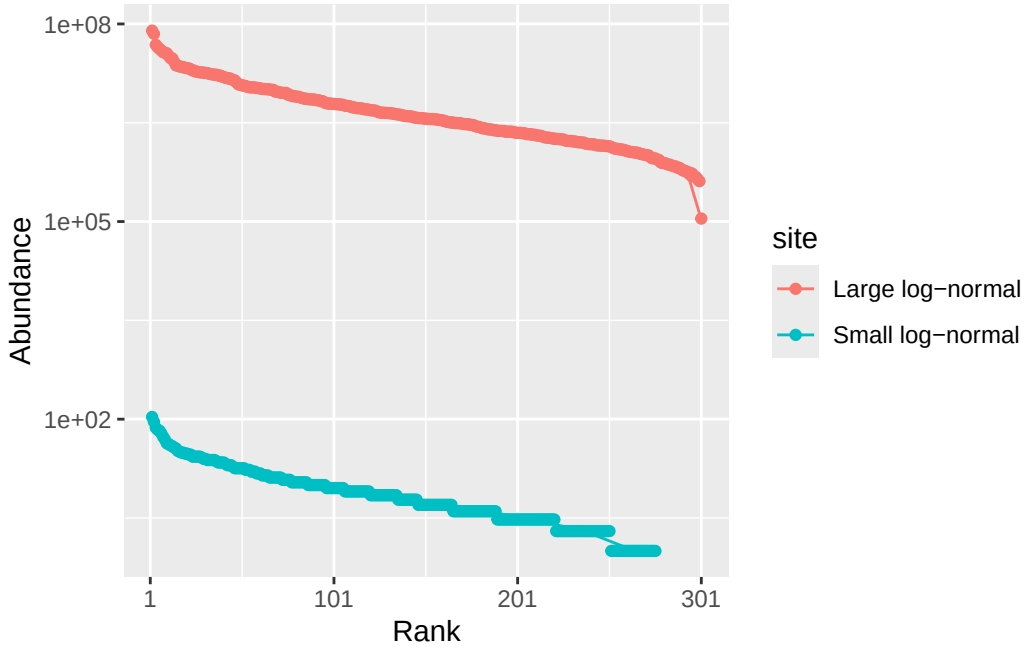


Big sample size : no species is unobserved.

Sample it :

```r
# Draw 3000 trees from the large community
rcommunity(
```

```
  1,
  size = 3000,
  abd = as.numeric(comm_large_lnorm)
) %>%
  # Name it
  mutate(site = "Small log-normal") ->
  comm_small_lnorm
# Make a single tibble with all data
comm_large_lnorm %>%
  # Make a single tibble
  bind_rows(comm_small_lnorm) ->
  comm_lnorm
# Plot the distributions and fit a lognormal model
autoplot(comm_lnorm, fit_rac = TRUE, distribution = "lnorm")
```



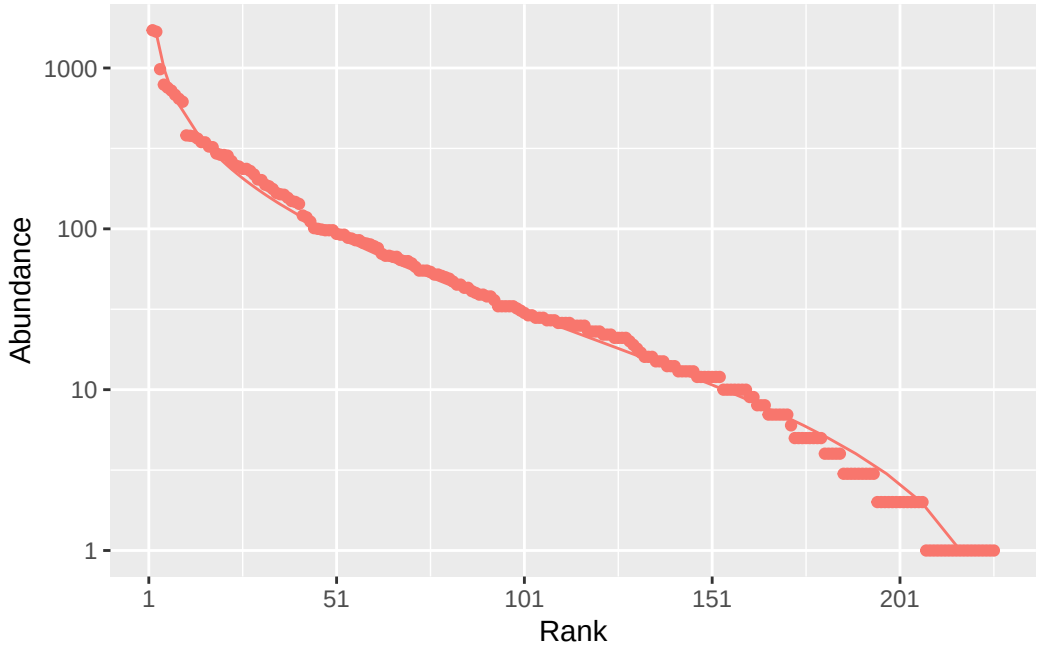

```
# Estimate the parameters
fit_rac(comm_lnorm, distribution = "lnorm")
```

```
# A tibble: 2 x 3
  site                mu sigma
  <chr>            <dbl> <dbl>
1 Large log-normal 15.2   1.10
```

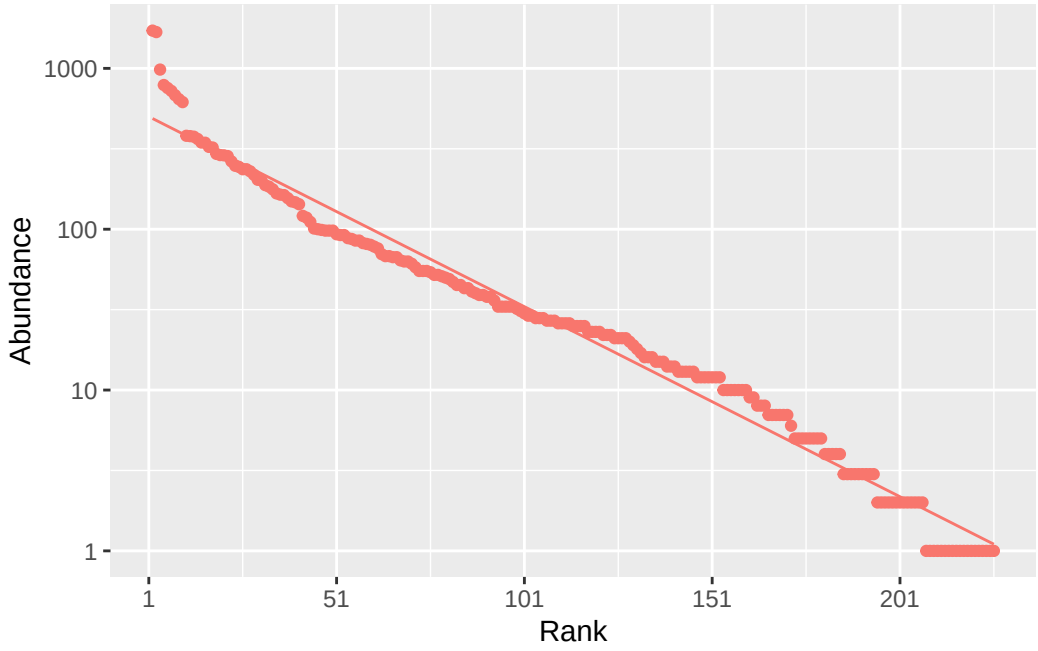
```
2 Small log-normal  1.83  1.05
```

Fit the distribition of BCI :

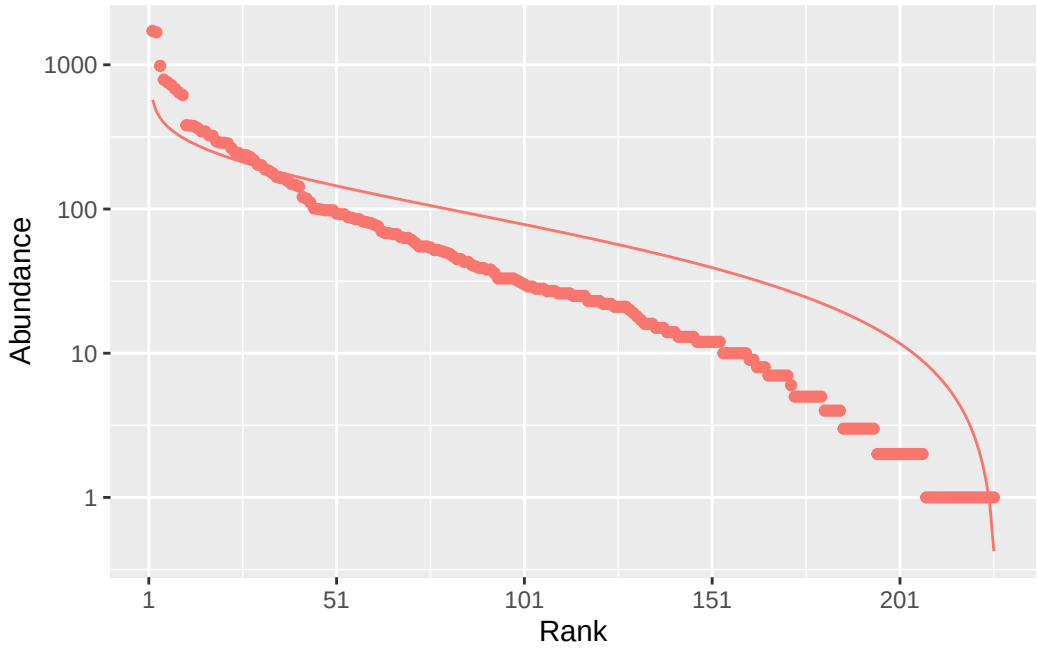
```
BCI_50ha_abd %>% as_abundances() %>% autoplot(fit_rac = TRUE, distribution="lnorm")
```



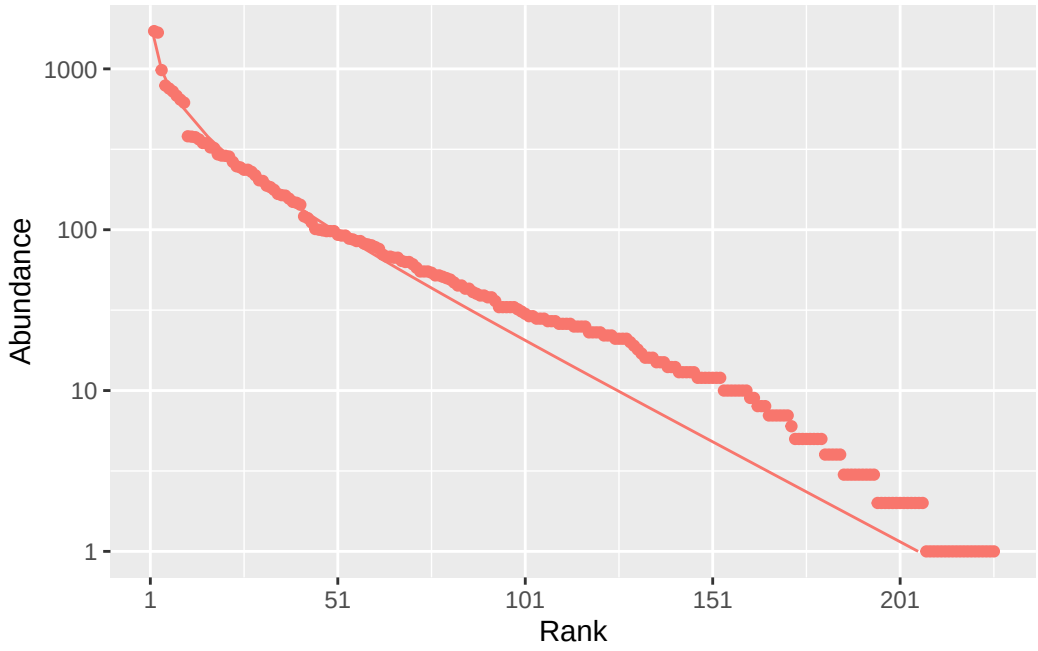

```
# We can try others distributions :
BCI_50ha_abd %>% as_abundances() %>% autoplot(fit_rac = TRUE, distribution="geom")
```




```
BCI_50ha_abd %>% as_abundances() %>% autoplot(fit_rac = TRUE, distribution="bstick")
```



```
BCI_50ha_abd %>% as_abundances() %>% autoplot(fit_rac = TRUE, distribution="lseries")
```



3.7 Zero-sum multinomial distribution

(nous on a déjà la file donc la partie du code qui calcule se lance pas)

BCI parameters θ and m are estimated following Etienne (2005) with the untb package.

```
# BCI data
library("vegan")
data(BCI)
BCI_50ha <- colSums(BCI)
# Don't load untb because of conflicts with dplyr::count() and select()
# library("untb")
if (file.exists("data/BCI_logkda.RData")) {
  # Use the saved result if available to save
  # computation resources
  load("data/BCI_logkda.RData")
} else {
  BCI_50ha %>%
  # name conflict with dplyr::count()
  untb::count() %>%
```

```
  # requires pari/gp installed and in the search paths,
  # or use gp_binary argument
  untb::logkda() ->
  BCI_logkda
  # Save the result
  save(BCI_logkda, file = "data/BCI_logkda.RData")
}
BCI_50ha %>%
  # name conflict with dplyr::count()
  untb::count() %>%
  # Estimate theta and m
  untb::optimal.params(log.kda = BCI_logkda) %>%
  print() ->
  BCI_params
```

```
     theta          m
47.6001882  0.0926531
```

theta = alpha et m=migration rate

Now we have the parameters of BCI, we simulate the community of BCI based on our parameters and the neutral theory :

The distribution of the local community is derived. J is that of BCI and m was estimated previously.

```
J <- sum(BCI)
theta <- BCI_params[1]
m <- BCI_params[2]
library("sads")
```

```
Le chargement a nécessité le package : stats4

Le chargement a nécessité le package : bbmle


Attachement du package : 'bbmle'

L'objet suivant est masqué depuis 'package:dplyr':

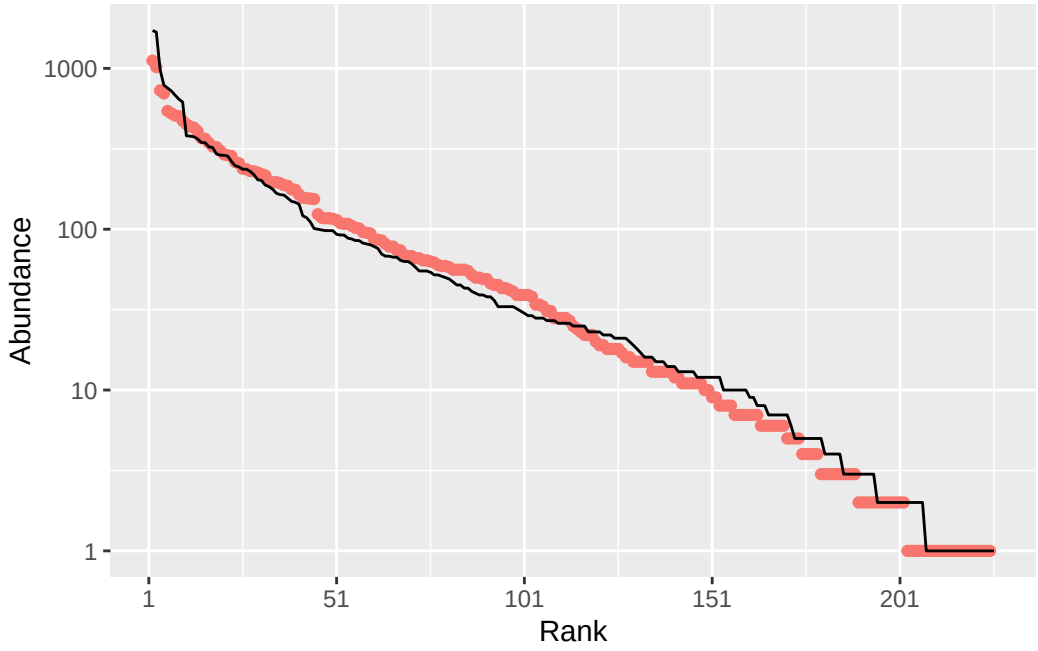
    slice
```

```
# Number of species in the local community
# random community. J is the size of the local community, theta & m are for the metacommunity
Svolkov(theta, m, J) %>%
  # Must be integers for the simulation
  round() %>%
  # Draw the species abundances
  rvolkov(theta, m, J) %>%
  # Make it an abundances object top plot it
  as_abundances() %>%
  # Whittaker plot
  autoplot() ->
  BCI_sim_neutral
```

```
Warning in Svolkov(theta, m, J): The use of this function outside sads core
code is deprecated and will be removed in a future version!
```

```
Warning in regularize.values(x, y, ties, missing(ties), na.rm = na.rm):
suppression des ex-aequos de 'x'
```

```
# BCI real distribution
BCI_50ha %>%
  sort(decreasing = TRUE) %>%
  as_tibble() %>%
  mutate(rank = seq_len(n())) ->
  BCI_tbl_50ha
# Plot altogether
BCI_sim_neutral +
  geom_line(
    data = BCI_tbl_50ha,
    aes(x = rank, y = value)
  )
```



black line = real abundances of BCI
red line = modeled abundances of BCI based on the parameters and neutral theory

Very hard way to simulate the distribution. The previous way (lognormal approximation) is usually preferred.

4 Species Accumulation Curves

Rarefaction curve = theoritical expectation of the accumulation curve (si on la fait un nombre infini de fois puis on prend la moyenne).

With the exact total number of species in the community, we can guess the number of species expected at a certain sample size

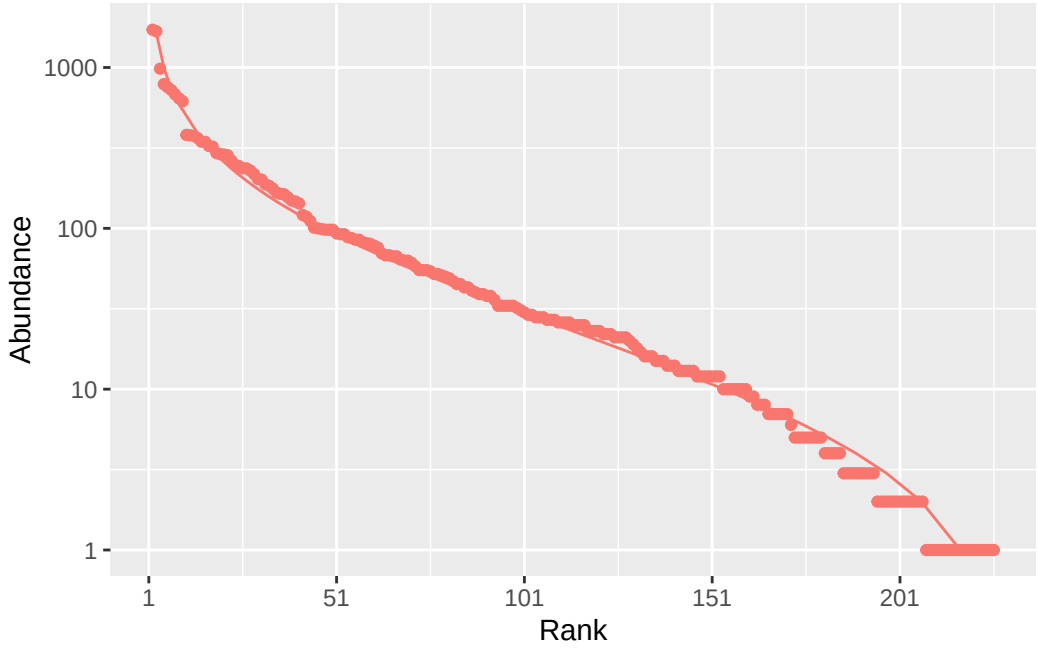
Multinomial distribution = set of binomial distributions that guarantee that the total number of individuals = sample size ? smth like that

Vague explication de la formule generalized Simpson entropy : probability to sample a new individual at each new sampled individual :
Ps(1-Ps)^n | Ps = probability to draw species S so 1-Ps = probability to draw another species than S and this probability n times (for n drawings) -> ^n

4.1 BCI data

Gather all subplots.

```
library("vegan")
data(BCI)
BCI %>%
  # Make an abundances object
  as_abundances() %>%
  # Make all plot weights equals
  mutate(weight = 1) %>%
  # Gather all 50 plots
  metacommunity() %>%
  # Name it
  mutate(site = "BCI") ->
  BCI_all
autoplot(BCI_all, fit_rac = TRUE, distribution = "lnorm")
```




```
fit_rac(BCI_all, distribution = "lnorm")
```

```
# A tibble: 1 x 3
  site     mu sigma
```

```
  <chr> <dbl> <dbl>
1 BCI    3.14  1.79
```

Abundance frequency counts help read the figure.

```
BCI %>%
colSums() %>%
abd_freq_count() %>%
arrange(desc(abundance))
```

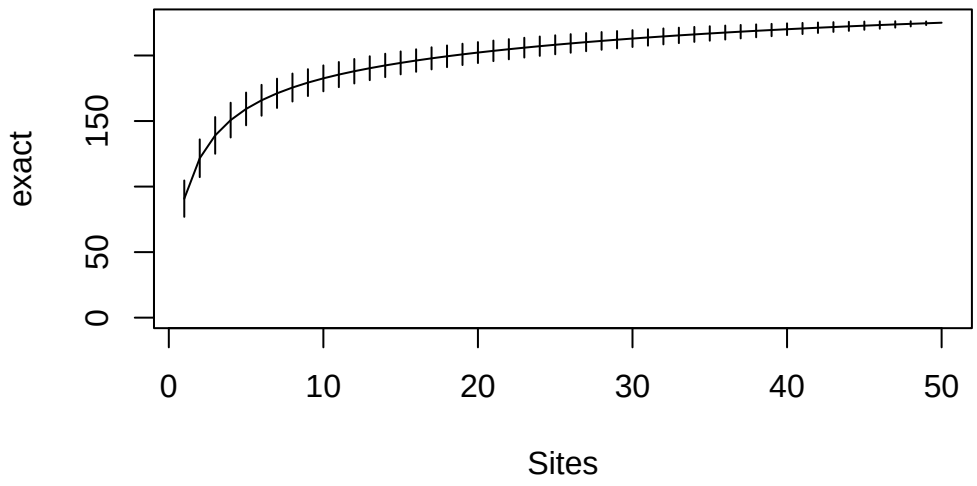
```
# A tibble: 108 x 2
   abundance number_of_species
       <dbl>         <int[1d]>
 1      1717                 1
 2      1681                 1
 3       983                 1
 4       788                 1
 5       755                 1
 6       724                 1
 7       681                 1
 8       644                 1
 9       617                 1
10       381                 1
# i 98 more rows
```

4.2 Accumulation of subplots

Accumulate subplots in random order (Monte-Carlo). Plot the expectation and the confidence interval of the number of species with respect to the number of samples.

```
# Plot par plot, pas par individu avec specaccum de vegan (et la manière dont le df de BCI es
# ça compute aussi l'intervalle de confiance
# on utilise plutôt la courbe par individu avec le package divent.+
BCI %>%
  specaccum() %>%
  plot()
```

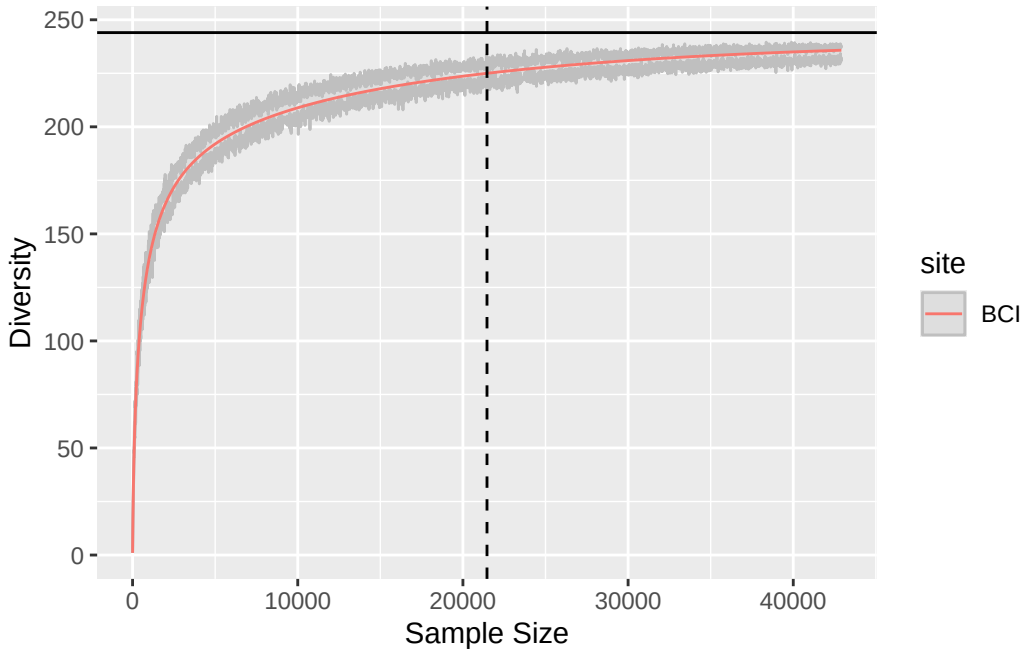
```
Warning in cor(x > 0): l'écart type est nul
```



4.3 Accumulation curve

Compute the expected accumulation curve of BCI up to twice the sampling effort, with its confidence interval.

```
# Number of trees in BCI
BCI_n <- abd_sum(BCI_all, as_numeric = TRUE)
# Asymptotic richness
BCI_richness <- div_richness(BCI_all, as_numeric = TRUE)
BCI_all %>%
  accum_hill(
    levels = seq(from = 1, to = 2 * BCI_n, by = 10),
    # Run simulations to estimate uncertainty.
    # Start with 10 simulations, run 100 later
    n_simulations = 10,
  ) %>%
  autoplot() +
  geom_vline(xintercept = BCI_n, linetype = 2) +
  geom_hline(yintercept = BCI_richness, linetype = 1)
```



4.4 Diapos 66-68

4.4.1 SAC per area :

nb d'individus dans une aire donnée = stochastique
suit une loi de Poisson
Donc quand on fait une courbe d'accumulation par aire on suppose que c'est proportionnel mais bon voilà quoi

4.4.2 Spatial structure of the forest :

species are aggregated
when doing field surveys, we sample trees close to each other (samples are not independant)
-> we cannot multiply the probabilities

4.4.3 Arrhenius's law

$$ln[S(A)] = lnc + zln(A)$$

S(A) : number of species depending on the area

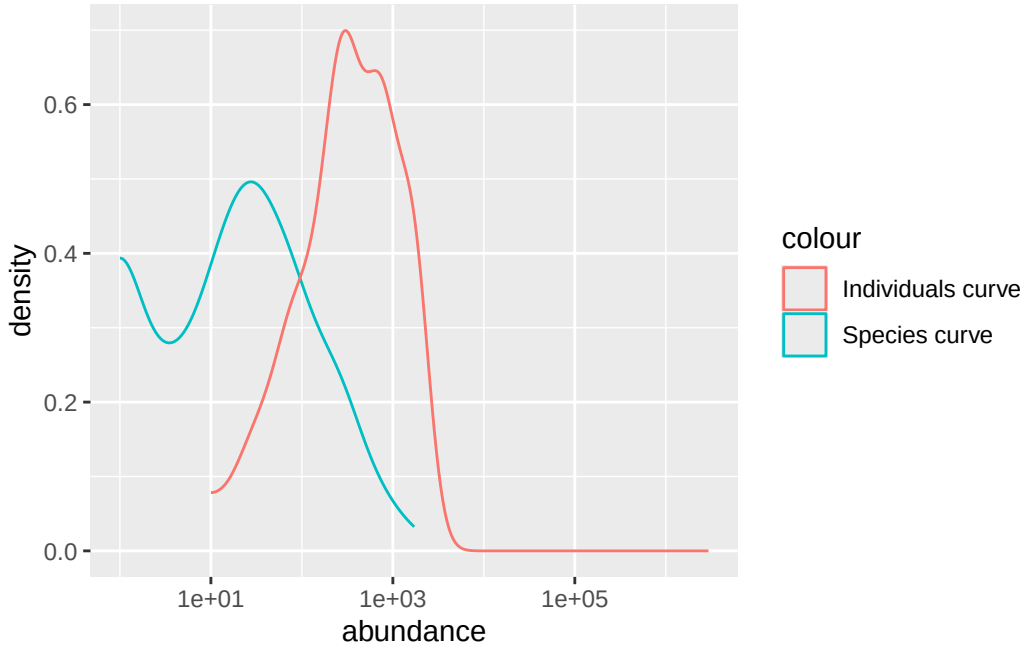
5 Canonical lognormal

Si l'abondance de l'espèce la plus abondante correspond au nombre d'individus max sur la courbe individus alors c'est une distribution lognormal canonique.

ggplot seems to bug with weighted, bounded density and log scale.

```
# Most adundant species
BCI_50ha_mas <- max(BCI_50ha_abd)
BCI_50ha_abd %>%
  tibble(abundance = .) %>%
  ggplot() +
  # Bounded density
  geom_density(
    aes(x = abundance, color = "Species curve"),
    bounds = c(0, Inf)
  ) +
  # Weighted density
  geom_density(
    aes(x = abundance, weight = abundance, color = "Individuals curve"),
    bounds = c(min(BCI_50ha_abd), max(BCI_50ha_abd))
  ) +
  scale_x_log10()
```

```
Warning: Some data points are outside of `bounds`. Removing them.
```



Correct curves.

```
library("GoFKernel")
```

```
Le chargement a nécessité le package : KernSmooth
```

```
KernSmooth 2.23 chargé
 Copyright M. P. Wand 1997-2009
```

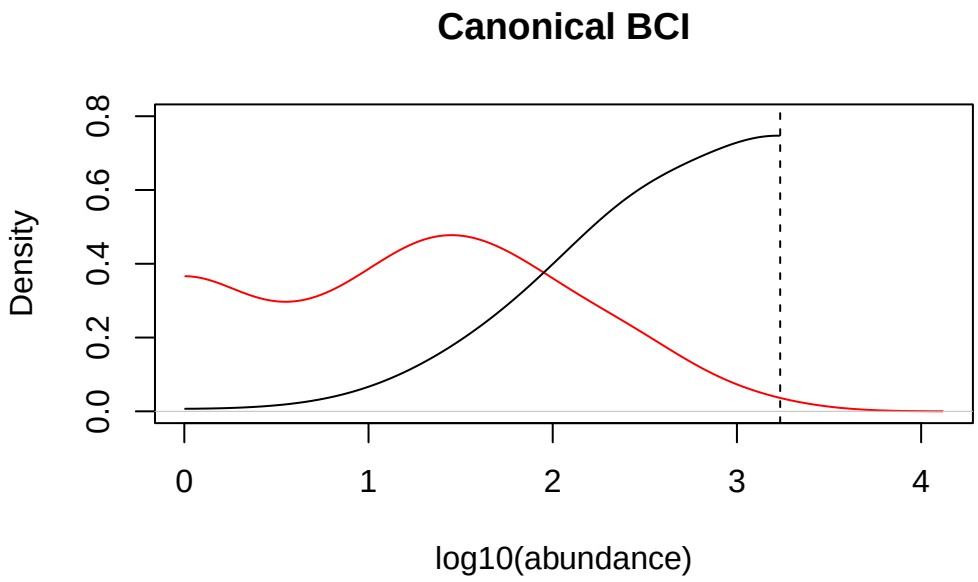
```
plot(
  density.reflected(log10(BCI_50ha_abd), lower = 0),
  type = "l",
  col = "red",
  ylim = c(0, .8),
  main = "Canonical BCI",
  xlab = "log10(abundance)"
)
lines(
  density.reflected(
    log10(BCI_50ha_abd),
    weights = BCI_50ha_abd,
    lower = min(log10(BCI_50ha_abd)),
```

```
    upper = max(log10(BCI_50ha_abd))
  )
)
```

```
Warning in density.default(x, weights = pesos, ...): Selecting bandwidth *not*
using 'weights'
```

```
Warning in density.default(x.reflect, weights = p.reflect, ...): Selecting
bandwidth *not* using 'weights'
```

```
abline(v = max(log10(BCI_50ha_abd)), lty = 2)
```



red curve : species curve : probabilty to encounter a species with the given abundance
black curve : individual curve : probability when you pull an individual to be part of a species that has the given abundance
Max of the black curve = max number of individuals. When it corresponds to the abundance of the most abundant species then it is cannonical lognormal. And then when you mix local scale and metacommunity? the distribution follows the power law.

6 II. Theory of the Measurement of Biodiversity

7 Richness

$$C = S_{obs}/S_{obs} + S_1$$

7.0.1 Jackknife estimators :

On considère que le nombre d'espèce que j'ai manqué S_0 = le nombre d'espèces que j'ai vu qu'une seule fois S_1.
Pour les espèces vues 1 fois : y'en a probablement beaucoup que j'aurais pu voir 2 fois en réalité. Donc je surestime S_1 donc je soustrais le nombre d'espèce que j'ai vu 2 fois. Etc. On a une succession de + (à chaque nombre impair) et de - (à chaque nombre pair).
Sample coverage = nb espèce échantillonnées / nb total d'espèce -> over 70% si on veut utiliser jackknife1.

7.1 Dummy communities

Build dummy communities.

```
tibble(
  sp_1 = c(10, 10, 1000, 1000),
  sp_2 = c(10, 11,    1,    0)
) %>%
  as_abundances() %>%
  print() ->
  comm_dummy
```

```
# A tibble: 4 x 4
  site   weight  sp_1  sp_2
  <chr>   <dbl> <dbl> <dbl>
1 site_1     20    10    10
2 site_2     21    10    11
3 site_3   1001  1000     1
4 site_4   1000  1000     0
```

Sample richness.

```
div_richness(comm_dummy, estimator = "naive")
```

```
# A tibble: 4 x 5
  site   weight estimator      order diversity
  <chr>   <dbl> <chr>          <dbl>     <dbl>
1 site_1     20 naive              0         2
2 site_2     21 naive              0         2
3 site_3   1001 naive              0         2
4 site_4   1000 Single Species     0         1
```

Asymptotic richness.

```
# on précise pas l'estimateur donc la fonction prend le meilleur
div_richness(comm_dummy)
```

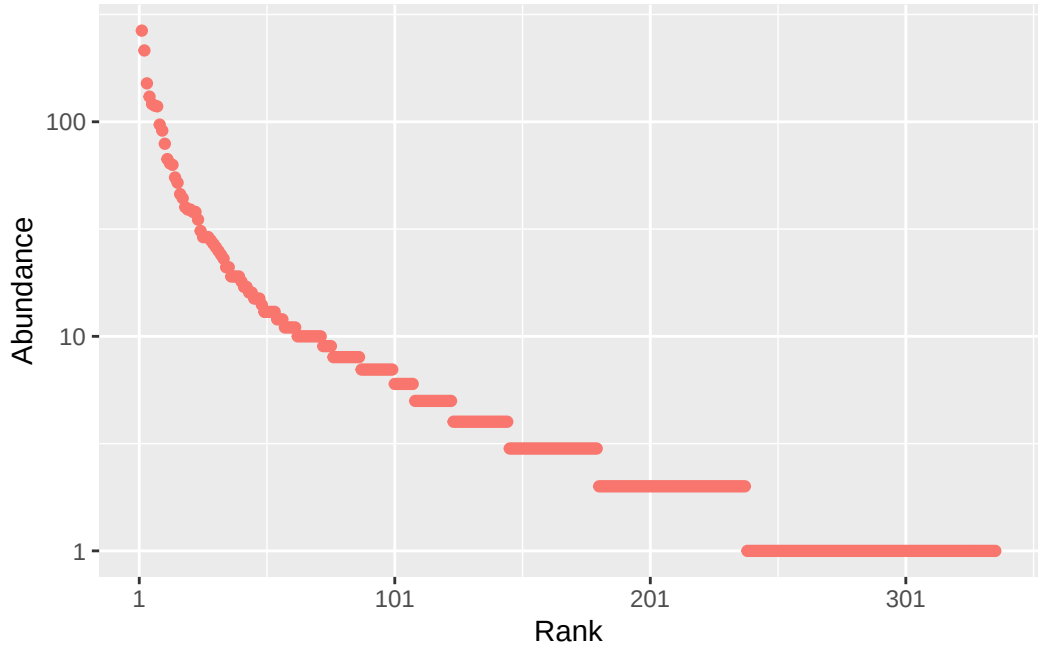
```
# A tibble: 4 x 5
  site   weight estimator      order diversity
  <chr>   <dbl> <chr>          <dbl>     <dbl>
1 site_1     20 Jackknife 0        0         2
2 site_2     21 Jackknife 1        0         2
3 site_3   1001 Jackknife 1        0         3
4 site_4   1000 Single Species     0         1
```

site_3 : 3 espèces prédites car il y a un singleton

7.2 Paracou 6

Prepare the data: gather the four subplots.

```
paracou_6_abd %>%
  # Gather all sites
  metacommunity() %>%
  # Rename "matacommunity"
  mutate(site = "Paracou 6") ->
  p6_abundances
autoplot(p6_abundances)
```



Abundance frequencies.

```
p6_abundances %>%
  as.numeric() %>%
  abd_freq_count()
```

```
# A tibble: 49 x 2
   abundance number_of_species
       <dbl>          <int[1d]>
 1         1                98
 2         2                58
 3         3                35
 4         4                22
 5         5                15
 6         6                 8
 7         7                13
 8         8                11
 9         9                 4
10        10                10
# i 39 more rows
```

```r
# pour lire les abondances décroissantes (pratique)
p6_abundances %>%
  as.numeric() %>%
  abd_freq_count() %>%
  arrange(desc(abundance))
```

```
# A tibble: 49 x 2
   abundance number_of_species
       <dbl>         <int[1d]>
 1       266                 1
 2       215                 1
 3       151                 1
 4       131                 1
 5       121                 1
 6       119                 1
 7       118                 1
 8        97                 1
 9        91                 1
10        79                 1
# i 39 more rows
```

Estimators of richness.

```r
div_richness(p6_abundances, estimator = "naive") %>%
  bind_rows(div_richness(p6_abundances, estimator = "Chao1")) %>%
  bind_rows(div_richness(p6_abundances, estimator = "iChao")) %>%
  # Default estimator
  bind_rows(div_richness(p6_abundances, estimator = "jackknife")) %>%
  bind_rows(div_richness(p6_abundances, estimator = "jackknife", jack_max = 1))
```

```
# A tibble: 5 x 5
  site      weight estimator   order diversity
  <chr>      <dbl> <chr>       <dbl>     <dbl>
1 Paracou 6   6.25 naive           0      335
2 Paracou 6   6.25 Chao1           0      418.
3 Paracou 6   6.25 iChao1          0      438.
4 Paracou 6   6.25 Jackknife 2     0      473
5 Paracou 6   6.25 Jackknife 1     0      433
```

7.3 Interpolation / extrapolation

Diversity of 1 ha in Paracou. Quand on interpole : on a la expected value qui est une valeur exacte (d'après la loi multinomiale bien connue, à gauche de la courbe d'accumulation).

```
# Number of trees per hectare
p6_density <- abd_sum(p6_abundances, as_numeric = TRUE) / 6.25
# Richness
p6_abundances %>%
  div_richness(level = round(p6_density))
```

```
# A tibble: 1 x 6
  site      weight estimator order level diversity
  <chr>      <dbl> <chr>     <dbl> <dbl>     <dbl>
1 Paracou 6   6.25 SAC           0   567      159.
```

```
# obtenue par raréfaction : on part du sample de 6ha dont on connait la richesse et on diminu
```

Diversity of 10 ha in Paracou. Quand on extrapole : on obtient une valeur approximée (droite de la courbe d'accumulation).

```
# Richness
p6_abundances %>%
  div_richness(level = round(10 * p6_density))
```

```
# A tibble: 1 x 6
  site      weight estimator order level diversity
  <chr>      <dbl> <chr>     <dbl> <dbl>     <dbl>
1 Paracou 6   6.25 jackknife     0  5666      383.
```

7.3.0.1 For comparing plots with different areas :

You compare the asymptotic diversity but if sampand it's okay. But if sampling isn't large enough (less than 5,10 ha of data) or asymptotic too large?, then extra/interpole the data to standardize to a certain area :
usually we take area = **twice the size of the smallest plot.**

8 Shannon

"Complexity" = nb of possible interactions of any length (n) given the probability of each species
= entropy

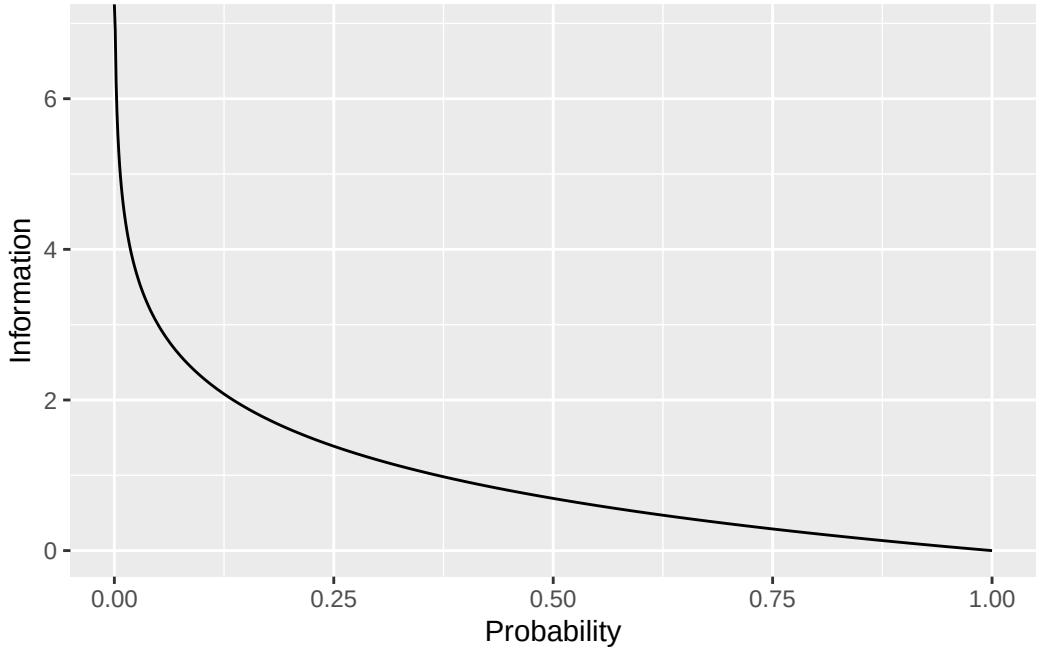
8.0.0.1 Definition of entropy :

"information" = how surprise am i (when meeting a common/rare species)
if p=0 then J=+infini
if p=1 then J=0 (no surprise)
Average information = weighted mean $(p_s * J(p_s))$
Rarity of a species = reciprocal of its probability -> 1/ps

8.1 Information function

Information brought by an individual: $I(p_s) = \ln(1/p_s)$.

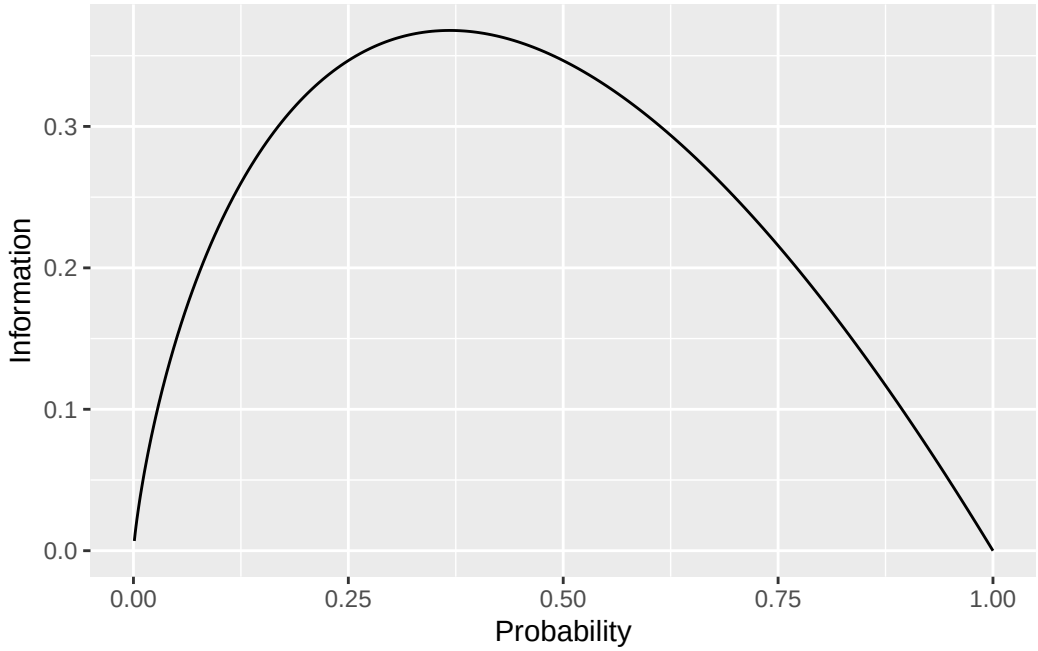
```
ggplot() +
  # Increase the default number of points (101 to 1001) for precision
  geom_function(fun = \(p) {log(1 / p)}, n = 1001) +
  labs(x = "Probability", y = "Information")
```



Information brought by a species: $p_s \ln(1/p_s)$.

```
ggplot() +
  geom_function(fun = \(p) {p * log(1 / p)}, n = 1001) +
  labs(x = "Probability", y = "Information")
```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_function()`).
```



Very rare species : very low probability. So when multiplying the information with the probability, the information isn't big enough to compensate for the very low probability -> courbe en cloche

8.1.0.1 Effective number of species :

Je considère que les S espèces ont la même probabilité 1/S, et je connais l'entropie de ma communité. S = effective number of species = nb d'espèces que j'ai avec cette entropie (et la même proba pour chacune) -> $S = e^H$
= minimum number of species for having that entropy

8.1.0.2 Estimators of entropy :

Espèces rares : si on les voit leur proba est surestimée et si on les manque leur proba est sous-estimée.
-> ChaoJost estimator : applique une correction par rapport au fait que les probas sont approximées.

-> UnveilJ : calcule de manière plus précise les probas (j'ai pas écouté comment) pour appliquer après la formule de base (c'est bien si on a trop de singletons auquel cas Chao1 ne convient pas, bref regarder diapo 34).

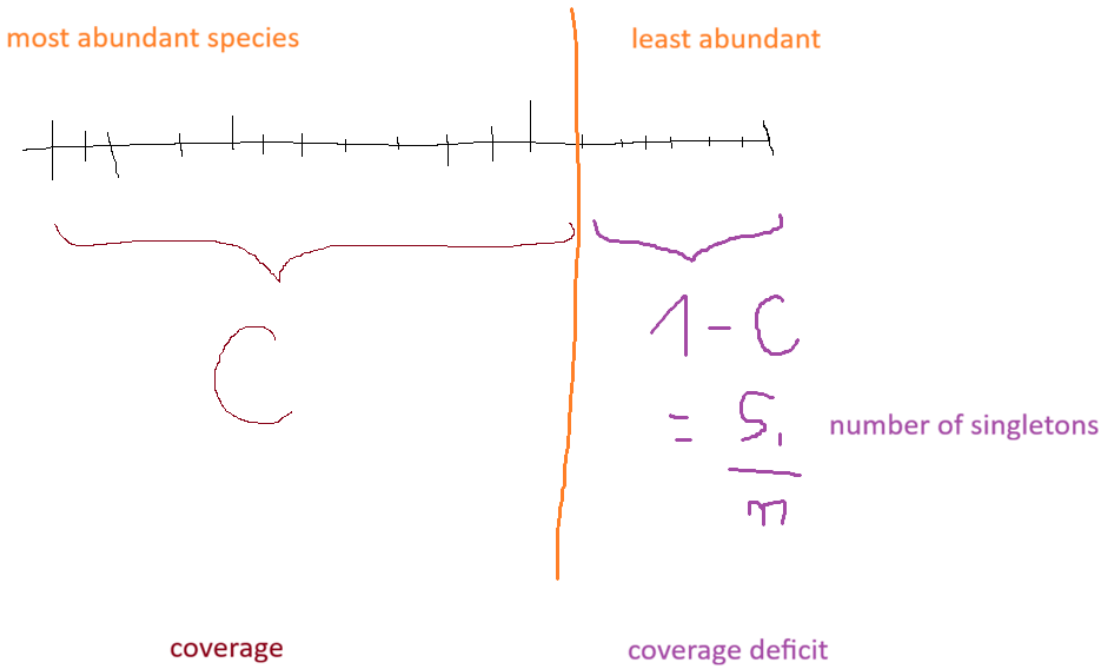


Figure 1: diapo 31 explanation

8.2 Paracou 6

Estimators.

```
ent_shannon(p6_abundances, estimator = "naive") %>%
  bind_rows(ent_shannon(p6_abundances, estimator = "UnveilJ")) %>%
  bind_rows(ent_shannon(p6_abundances, estimator = "ChaoJost")) %>%
```

```
  print() ->
  p6_shannon
```

```
# A tibble: 3 x 5
  site      weight estimator order entropy
  <chr>      <dbl> <chr>     <dbl>   <dbl>
1 Paracou 6   6.25 naive         1    4.62
2 Paracou 6   6.25 UnveilJ       1    4.71
3 Paracou 6   6.25 ChaoJost      1    4.68
```

Effective number of species: e^H.

```
p6_shannon %>%
  mutate(diversity = exp(entropy))
```

```
# A tibble: 3 x 6
  site      weight estimator order entropy diversity
  <chr>      <dbl> <chr>     <dbl>   <dbl>     <dbl>
1 Paracou 6   6.25 naive         1    4.62      101.
2 Paracou 6   6.25 UnveilJ       1    4.71      111.
3 Paracou 6   6.25 ChaoJost      1    4.68      108.
```

8.3 Dummy communities

Compare entropy.

```
comm_dummy
```

```
# A tibble: 4 x 4
  site   weight  sp_1  sp_2
  <chr>   <dbl> <dbl> <dbl>
1 site_1     20    10    10
2 site_2     21    10    11
3 site_3   1001  1000     1
4 site_4   1000  1000     0
```

```
ent_shannon(comm_dummy, estimator = "naive")
```

```
# A tibble: 4 x 5
  site   weight estimator      order entropy
  <chr>   <dbl> <chr>          <dbl>   <dbl>
1 site_1     20 naive              1 0.693
2 site_2     21 naive              1 0.692
3 site_3   1001 naive              1 0.00790
4 site_4   1000 Single Species     1 0
```

9 Simpson

Ne prend pas en compte les espèces rares (aucun poids dans la formule). Peu de surprise quand on draw une espèce rare.

9.1 Paracou 6

Estimators.

```
ent_simpson(p6_abundances, estimator = "naive") %>%
  bind_rows(ent_simpson(p6_abundances, estimator = "Lande")) %>%
  bind_rows(ent_simpson(p6_abundances, estimator = "UnveilJ")) %>%
  bind_rows(ent_simpson(p6_abundances, estimator = "ChaoJost")) %>%
  print() ->
  p6_simpson
```

```
# A tibble: 4 x 5
  site      weight estimator order entropy
  <chr>      <dbl> <chr>     <dbl>   <dbl>
1 Paracou 6   6.25 naive         2   0.978
2 Paracou 6   6.25 Lande         2   0.979
3 Paracou 6   6.25 UnveilJ       2   0.979
4 Paracou 6   6.25 ChaoJost      2   0.979
```

Effective number of species: $1/(1-E)$.

```
p6_simpson %>%
  mutate(diversity = 1 / (1 - entropy))
```

```
# A tibble: 4 x 6
  site      weight estimator order entropy diversity
  <chr>      <dbl> <chr>     <dbl>   <dbl>     <dbl>
1 Paracou 6   6.25 naive         2   0.978      45.9
2 Paracou 6   6.25 Lande         2   0.979      46.5
3 Paracou 6   6.25 UnveilJ       2   0.979      46.5
4 Paracou 6   6.25 ChaoJost      2   0.979      46.5
```

10 Generalized entropy

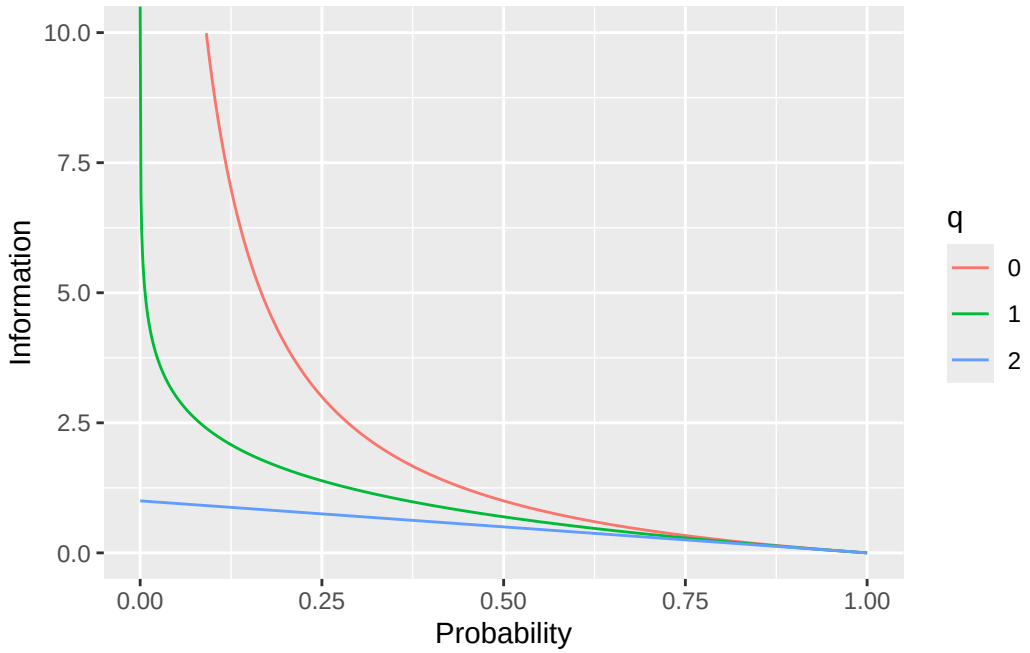
La sensibilité de la surprise détermine la mesure qu'on utilise (richness, Shannon or Simpson). Le paramètre q dans la formule généralisée dit à quel point on s'intéresse aux espèces rares : plus q est grand moins on s'intéresse aux espèces rares.

0 = number of species
D1 = diversity according to Shannon
D2 = diversity according to Simpson
Si on va vers la droite du graph = q augmente = on fait de moins en moins attention aux espèces rares :
D_infini = 1/p_max (plus qu'une espèce).

10.1 Information

Information brought by an individual: $I(p_s) = \ln_q(1/p_s)$.

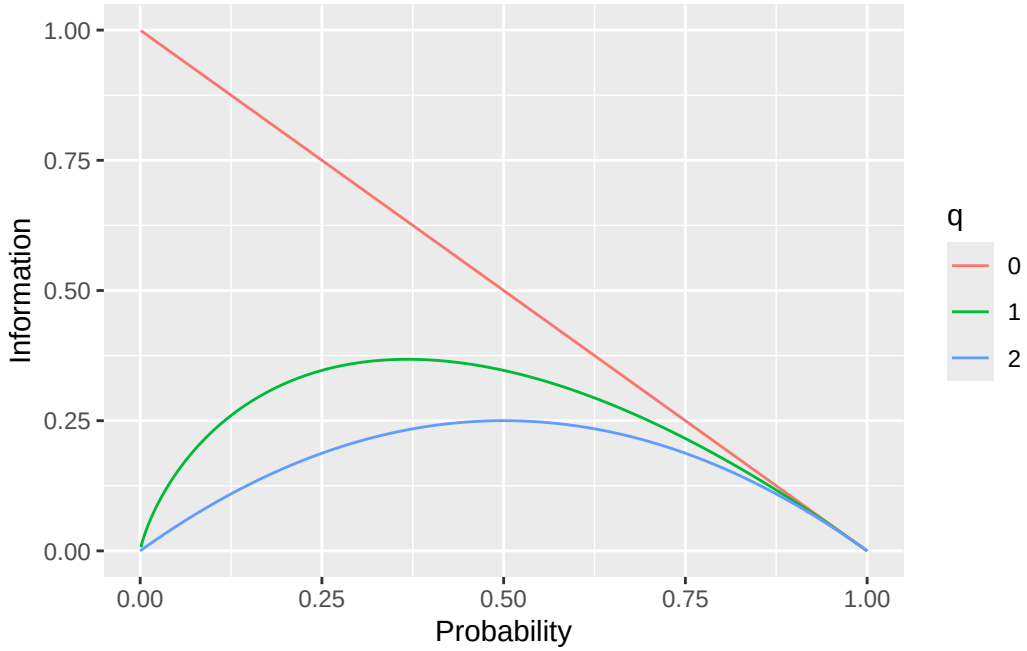
```
ggplot() +
  # Increase the default number of points (101 to 1001) for precision
  geom_function(fun = \(p) {ln_q(1 / p, q = 0)}, n = 1001, aes(color = "0")) +
  geom_function(fun = \(p) {ln_q(1 / p, q = 1)}, n = 1001, aes(color = "1")) +
  geom_function(fun = \(p) {ln_q(1 / p, q = 2)}, n = 1001, aes(color = "2")) +
  scale_y_continuous(limits = c(0, 10)) +
  labs(x = "Probability", y = "Information", color = "q")
```



Information brought by a species: $p_s \ln_q(1/p_s)$.

```
ggplot() +
  # Increase the default number of points (101 to 1001) for precision
  geom_function(fun = \(p) {p * ln_q(1 / p, q = 0)}, n = 1001, aes(color = "0")) +
  geom_function(fun = \(p) {p * ln_q(1 / p, q = 1)}, n = 1001, aes(color = "1")) +
  geom_function(fun = \(p) {p * ln_q(1 / p, q = 2)}, n = 1001, aes(color = "2")) +
  labs(x = "Probability", y = "Information", color  = "q")
```

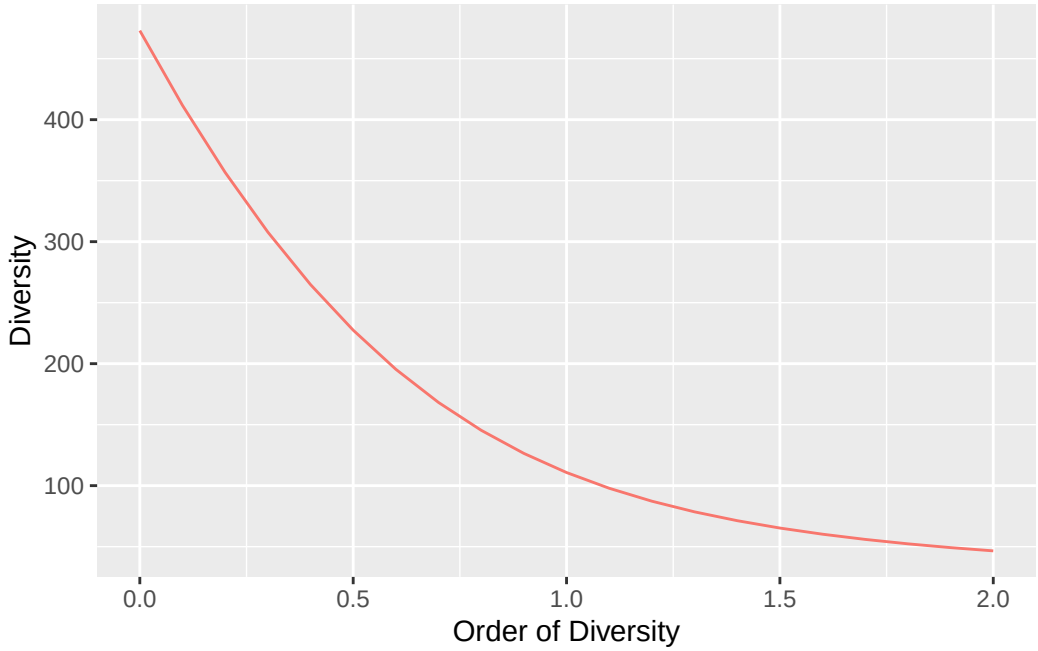
```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_function()`).
Removed 1 row containing missing values or values outside the scale range
(`geom_function()`).
```



10.2 Paracou 6

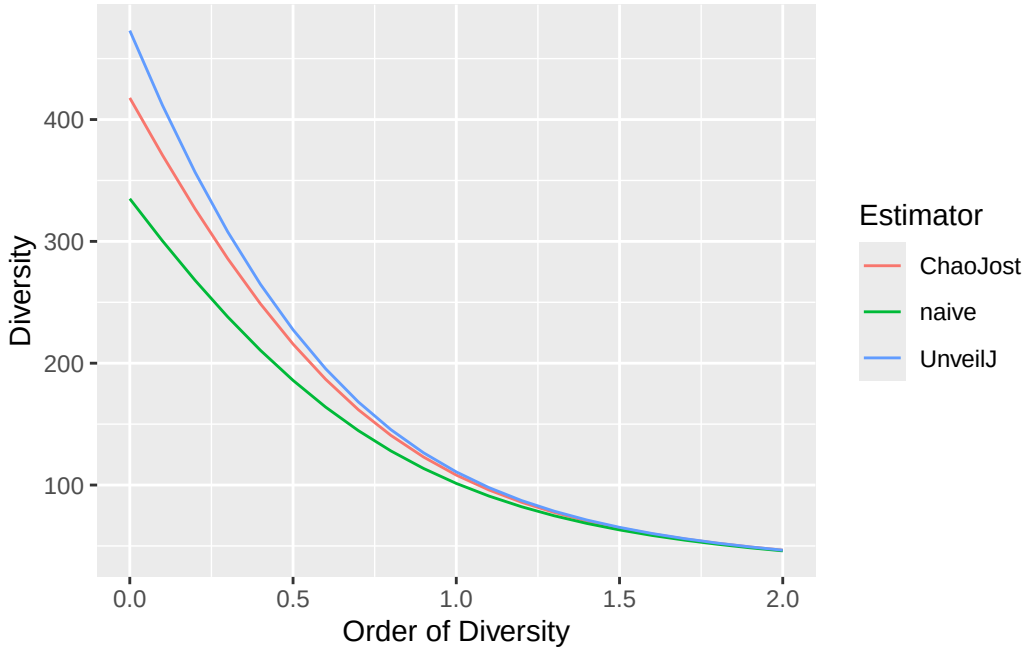
Diversity profile.

```
p6_abundances %>%
  profile_hill() %>%
  autoplot() +
  # Don't print the legend
  theme(legend.position = "none")
```



Compare estimators.

```
p6_abundances %>%
  profile_hill(estimator = "UnveilJ") %>%
  # Use the site name to store the estimator's name
  mutate(site = "UnveilJ") %>%
  bind_rows(
    p6_abundances %>%
    profile_hill(estimator = "ChaoJost") %>%
    mutate(site = "ChaoJost")
  ) %>%
  bind_rows(
    p6_abundances %>%
    profile_hill(estimator = "naive") %>%
    mutate(site = "naive")
  ) %>%
  autoplot() +
  labs(color  = "Estimator")
```



Extreme values.

The reciprocal of the Berger-Parker index (Berger and Parker 1970) is the rarity of the most abundant species ${}^{+\infty}D$.

```
# q = 1000 is enough for infinity
div_hill(p6_abundances, q = 1000)
```

```
# A tibble: 1 x 5
  site      weight estimator order diversity
  <chr>      <dbl> <chr>     <dbl>     <dbl>
1 Paracou 6   6.25 UnveilJ    1000      13.3
```

```
# Compare to
p6_abundances %>%
  # Extract proobabilities
  as_probabilities(as_numeric = TRUE) %>%
  # Max value
  max() %>%
  # Reciprocal
  `^`(-1)
```

```
[1] 13.31203
```

10.3 Dummy communities

Compare diversity

```
comm_dummy
```

```
# A tibble: 4 x 4
  site   weight  sp_1  sp_2
  <chr>   <dbl> <dbl> <dbl>
1 site_1     20    10    10
2 site_2     21    10    11
3 site_3   1001  1000     1
4 site_4   1000  1000     0
```

```
div_hill(comm_dummy, q = 1, estimator = "naive")
```

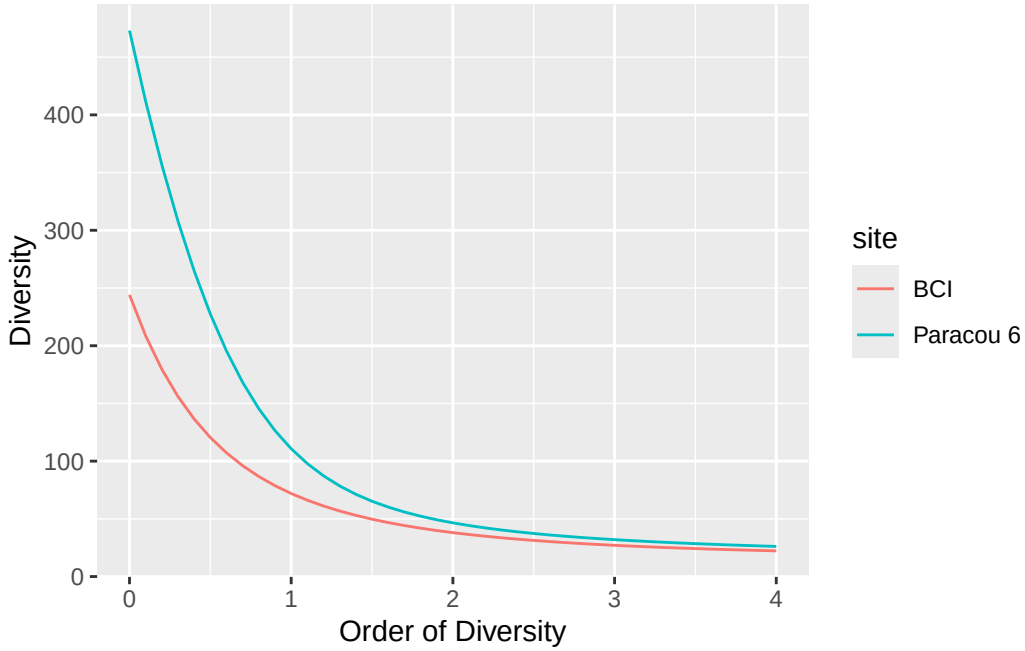
```
# A tibble: 4 x 5
  site   weight estimator      order diversity
  <chr>   <dbl> <chr>          <dbl>     <dbl>
1 site_1     20 naive              1      2
2 site_2     21 naive              1      2.00
3 site_3   1001 naive              1      1.01
4 site_4   1000 Single Species     1      1
```

11 Comparing diversity

11.1 Asymptotic diversity

Compare asymptotic diversity of Paracou and BCI, extend q.

```
# Orders from 0 to 4
q_seq_0_4 <- seq(from = 0, to = 4, by = 0.1)
profile_hill(p6_abundances, orders = q_seq_0_4) %>%
  bind_rows(profile_hill(BCI_all, orders = q_seq_0_4)) %>%
  autoplot()
```



11.2 Standardized diversity

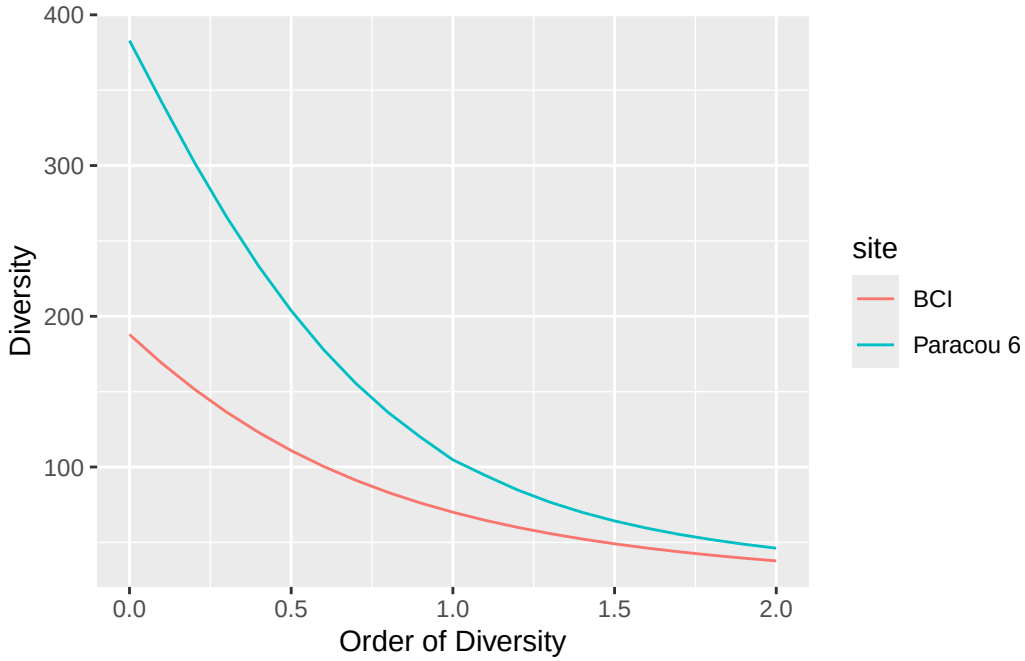
The sampling effort is very different: 50 ha in BCI, 6.25 ha in Paracou. The asymptotic estimation is less reliable in Paracou.

Compare diversity at the same sampling effort, say 10 ha.

```
# Number of trees in 10 ha
p6_n_10ha <- round(abd_sum(p6_abundances, as_numeric = TRUE) / 6.25 * 10)
BCI_n_10_ha <- round(sum(BCI_all) / 5)
```

Compare diversity of 10 ha.

```
profile_hill(p6_abundances, level = p6_n_10ha) %>%
  bind_rows(profile_hill(BCI_all, level = BCI_n_10_ha)) %>%
  autoplot()
```



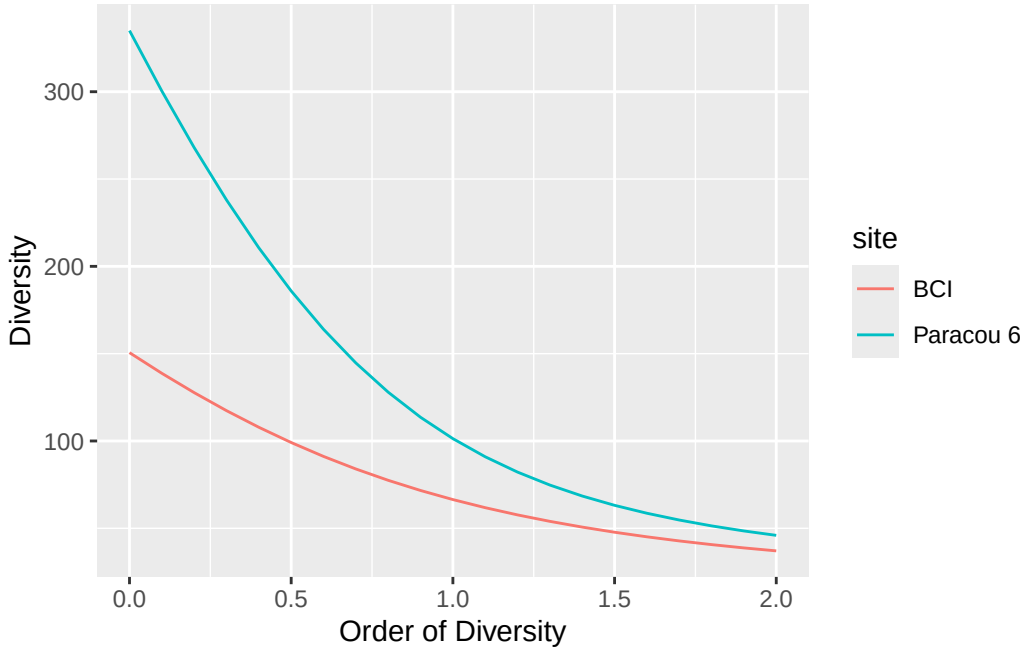
11.3 Sample coverage

```
coverage(p6_abundances) %>%
  bind_rows(coverage(BCI_all))
```

```
# A tibble: 2 x 4
  site      weight estimator  coverage
  <chr>      <dbl> <chr>         <dbl>
1 Paracou 6   6.25 ZhangHuang    0.972
2 BCI        50    ZhangHuang    0.999
```

It's usually better to compare diversity at the same sample coverage (Chao and Jost 2012), instead of the area (arbres plus petits -> + d'arbres par unité de surface).

```
reference_coverage <- coverage(p6_abundances, as_numeric = TRUE)
profile_hill(p6_abundances, level = reference_coverage) %>%
  bind_rows(profile_hill(BCI_all, level = reference_coverage)) %>%
  autoplot()
```



12 Phylogenetic diversity

La formule horrible : on cut à chaque fois que ya une spéciation puis on calcule l'entropie pour chaque temps k et on moyenne l'entropie générale

12.1 Mock data

A simple community with 3 species.

```
mock_3sp_abd %>%
  as_abundances() %>%
  mutate(site = "Mock 3 species") %>%
  print() ->
  mock_3sp_abundances
```

```
# A tibble: 1 x 5
  site           weight     A     B     C
  <chr>           <dbl> <dbl> <dbl> <dbl>
1 Mock 3 species      6     3     2     1
```

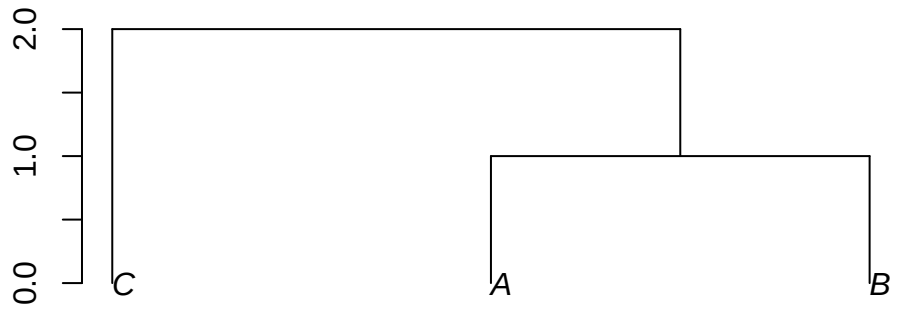
```
library("ape")
```

```
Attachement du package : 'ape'
```

```
L'objet suivant est masqué depuis 'package:dplyr':
```

```
    where
```

```
# Prepare the tree for convenience: make the branches twice longer
mock_3sp_tree$edge.length <- 2 * mock_3sp_tree$edge.length
plot(mock_3sp_tree, direction = "downwards", srt = 90)
axis(2)
```



PD is the total length of branches

```
div_pd(mock_3sp_abd, tree = mock_3sp_tree)
```

```
# A tibble: 1 x 3
  estimator order diversity
  <chr>     <dbl>     <dbl>
1 naive         0         5
```

Rao.

```
ent_rao(mock_3sp_abundances, tree = mock_3sp_tree, estimator = "naive")
```

```
# A tibble: 1 x 5
  site           weight estimator order entropy
  <chr>           <dbl> <chr>     <dbl>   <dbl>
1 Mock 3 species      6 naive         2   0.528
```

The tree is normalized to height = 1 by default. Without normalization: here it doubles

```
ent_rao(
  mock_3sp_abundances,
  tree = mock_3sp_tree,
  estimator = "naive",
  normalize = FALSE
)
```

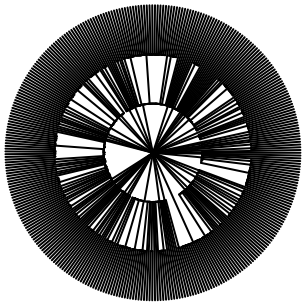
```
# A tibble: 1 x 5
  site           weight estimator order entropy
  <chr>           <dbl> <chr>     <dbl>   <dbl>
1 Mock 3 species      6 naive         2    1.06
```

The entropy is the average length along the tree -> good idea to normallize.

12.2 Paracou 6

`paracou_6_taxo` is a taxonomic tree: distance is 1 between species of the same genus, 2 between genera of the same family, 3 between families.

```
plot(paracou_6_taxo, type = "fan", show.tip.label = FALSE)
```



Calculate Rao's entropy and phylogenetic entropy of order 2

```
# rao's way : average along the tree
ent_rao(p6_abundances, tree = paracou_6_taxo) %>%
  bind_rows(
    ent_phylo(p6_abundances, tree = paracou_6_taxo, q = 2)
  )
```

```
# A tibble: 2 x 6
  site      weight estimator order entropy     q
  <chr>      <dbl> <chr>     <dbl>   <dbl> <dbl>
1 Paracou 6   6.25 Lande         2   0.976    NA
2 Paracou 6   6.25 UnveilJ      NA   0.949     2
```

```
#ent_phylo : general way : cuts the tree in slices then computes the simpson entropy in each
```

High entropy value : high probability that 2 drawn species are completely different.

Rao's entropy is not an effective number of species, but the average phylogenetic distance between two individuals. **The corresponding Hill number is:**

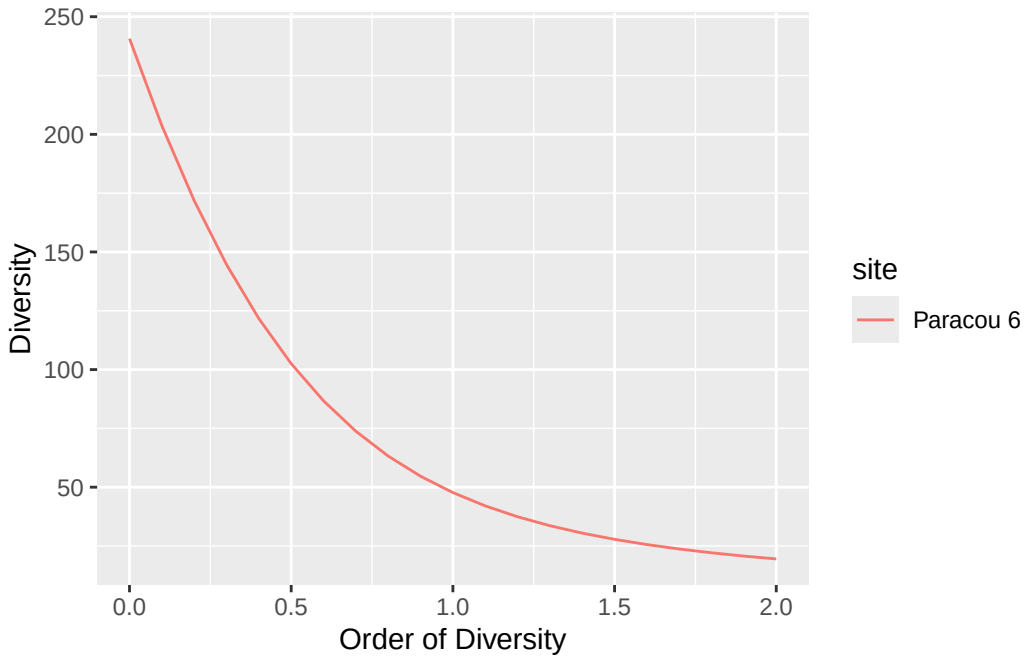
```
# fonction pour le hill number :
div_phylo(p6_abundances, tree = paracou_6_taxo, q = 2)
```

```
# A tibble: 1 x 4
  site      weight estimator diversity
  <chr>      <dbl> <chr>         <dbl>
1 Paracou 6   6.25 UnveilJ        19.5
```

Nombre d'espèces qu'on aurait avec cette entropie si elles étaient toutes à la même distance de leur ancêtre commun (chacune a 1 seul ancêtre commun).

Diversity profile

```
profile_phylo(p6_abundances, tree = paracou_6_taxo) %>%
  autoplot()
```



13 Functional diversity

13.1 Mock data

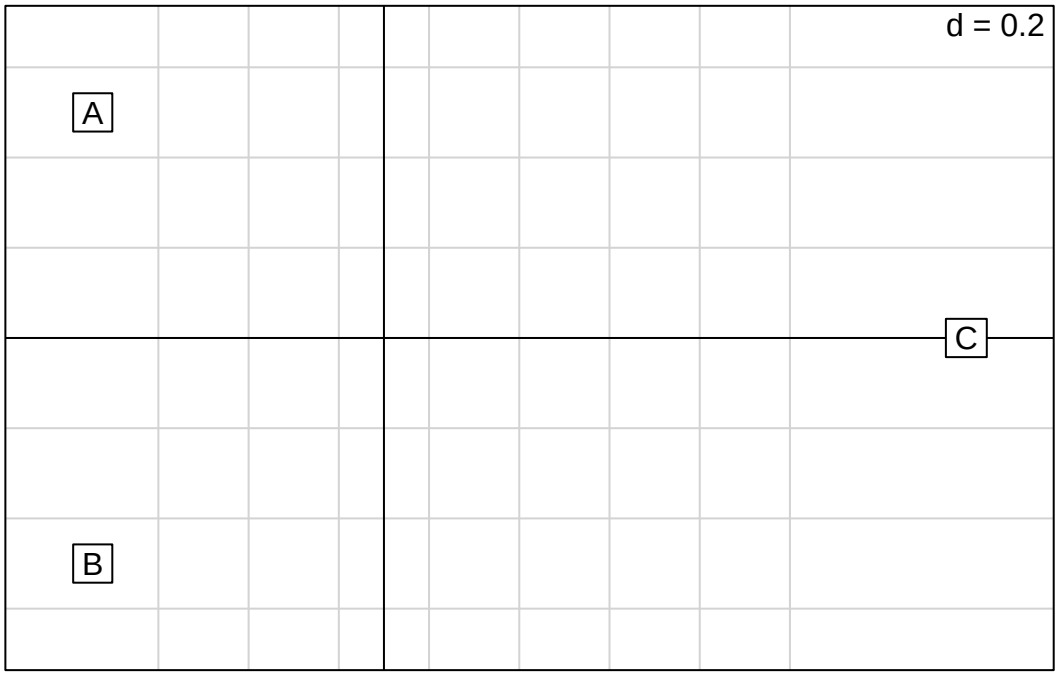
The same mock data is used. Distances between species is now assumed to be in the trait space. Distances between species.

```
mock_3sp_dist
```

```
  A B
B 1
C 2 2
```

Plot the trait space.

```
library("ade4")
mock_pcoa <- dudi.pco(mock_3sp_dist, scannf = FALSE)
scatter(mock_pcoa, posieig = "none")
```




```
NULL
```

Similarity between species.

```
# The dist object must be converted to a matrix
mock_3sp_Z <- 1 - as.matrix(mock_3sp_dist) / max(mock_3sp_dist)
mock_3sp_Z
```

```
    A   B C
A 1.0 0.5 0
B 0.5 1.0 0
C 0.0 0.0 1
```

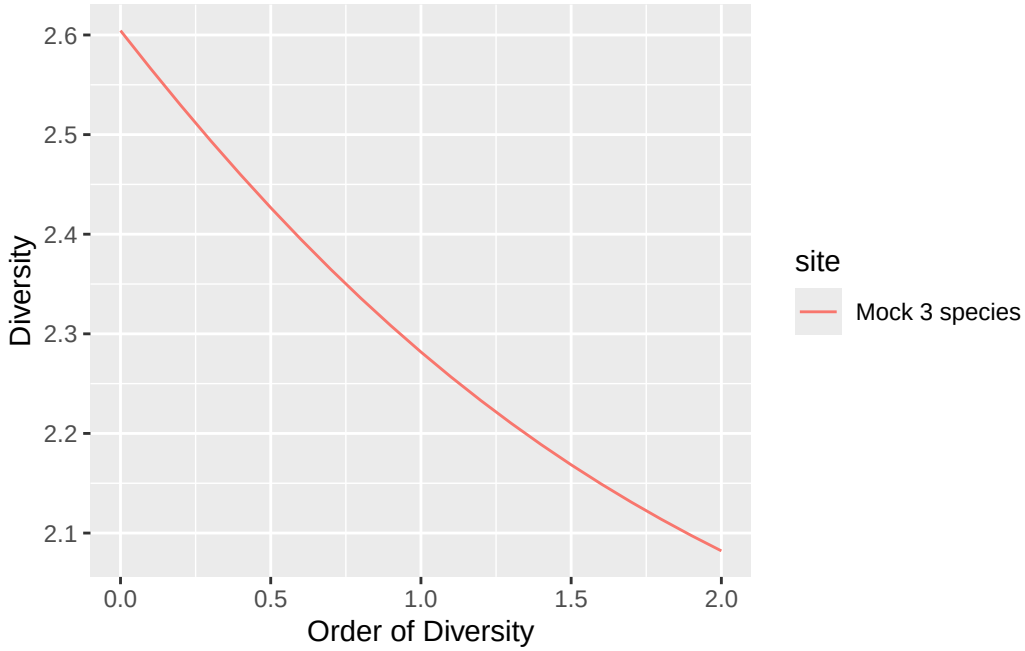
Ordinariness.

```
# Probabilities of species
mock_3sp_prob <- mock_3sp_abd / sum(mock_3sp_abd)
# Ordinariness of species
mock_3sp_O <- as.numeric(mock_3sp_Z %*% mock_3sp_prob)
# Compare probabilities and ordinariness
tibble(
  Probability = mock_3sp_prob,
  Rarity_p = 1 / mock_3sp_prob,
  Ordinariness = mock_3sp_O,
  Rarity_O = 1 / mock_3sp_O
)
```

```
# A tibble: 3 x 4
  Probability Rarity_p Ordinariness Rarity_O
        <dbl>    <dbl>        <dbl>    <dbl>
1       0.5          2        0.667     1.5
2       0.333        3        0.583     1.71
3       0.167        6        0.167     6
```

Diversity.

```
profile_similarity(
  mock_3sp_abundances,
  similarities = mock_3sp_Z,
  estimator = "naive"
) %>%
  autoplot()
```



Diversity of order 2 (i.e. Rao's).

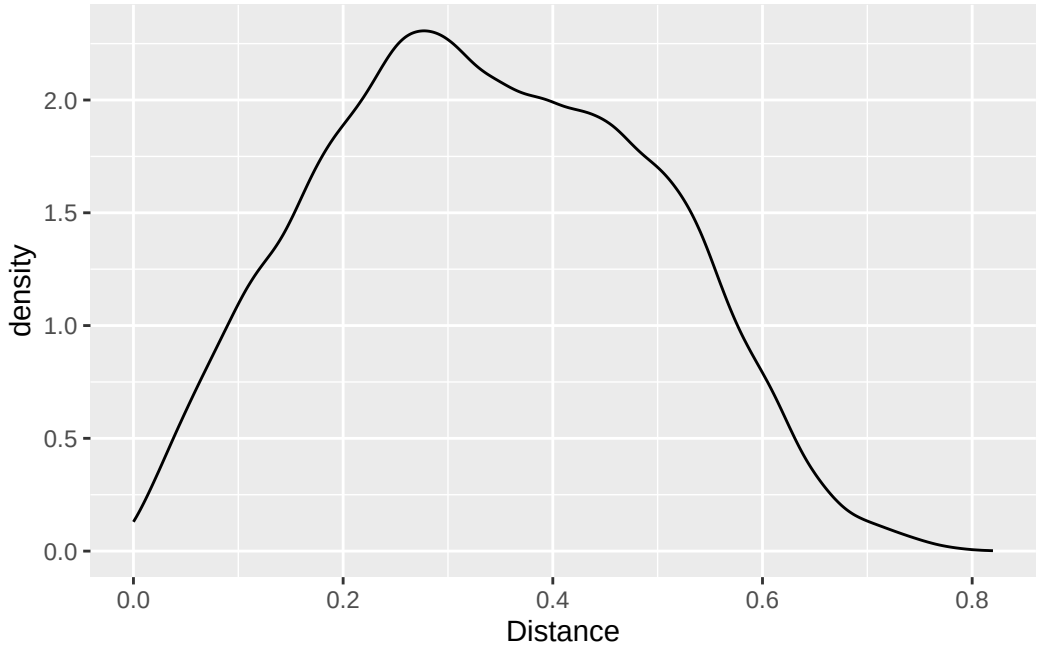
```
ent_similarity(
  mock_3sp_abundances,
  similarities = mock_3sp_Z,
  q = 2,
  estimator = "naive"
)
```

```
# A tibble: 1 x 5
  site           weight estimator order entropy
  <chr>           <dbl> <chr>     <dbl>   <dbl>
1 Mock 3 species      6 naive         2   0.444
```

13.2 Paracou 6

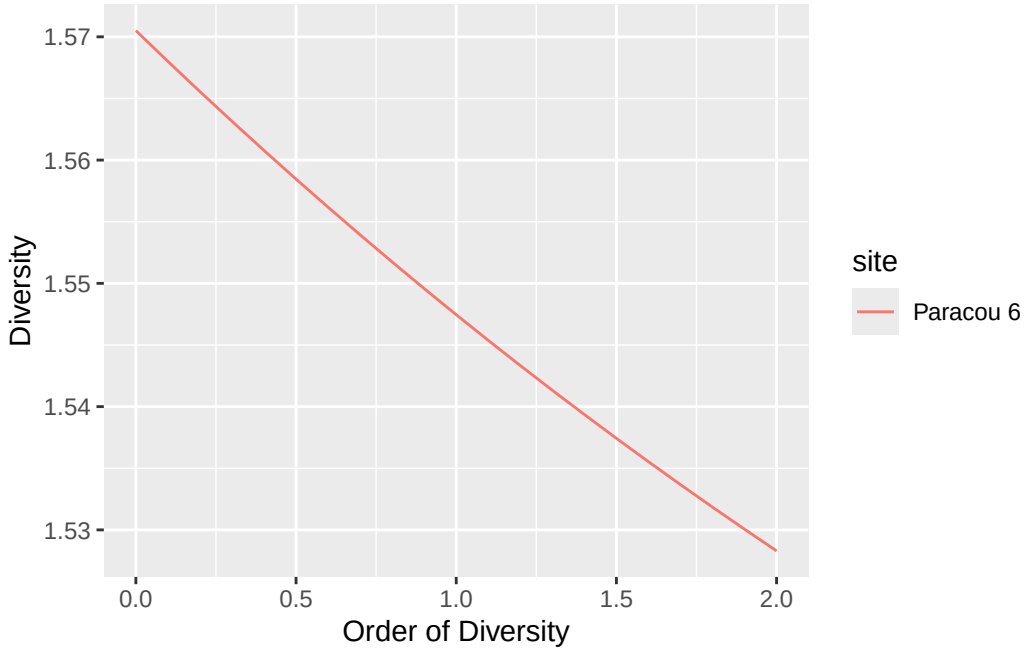
The species traits are maximum height (95th percentile) and seed mass, maximum height (95th percentile) and seed mass. A distance matrix was built (see home practical). Distribution of distances between species.

```
paracou_6_fundist %>%
  as.numeric() %>%
  tibble(Distance = .) %>%
  ggplot() +
  geom_density(aes(x = Distance))
```



Diversity profile.

```
p6_abundances %>%
  profile_similarity(
    similarities = 1 - as.matrix(paracou_6_fundist) / max(paracou_6_fundist)
  ) %>%
  autoplot()
```



13.3 Intraspecific variability

Coordinates of species A, B and C in the trait space.

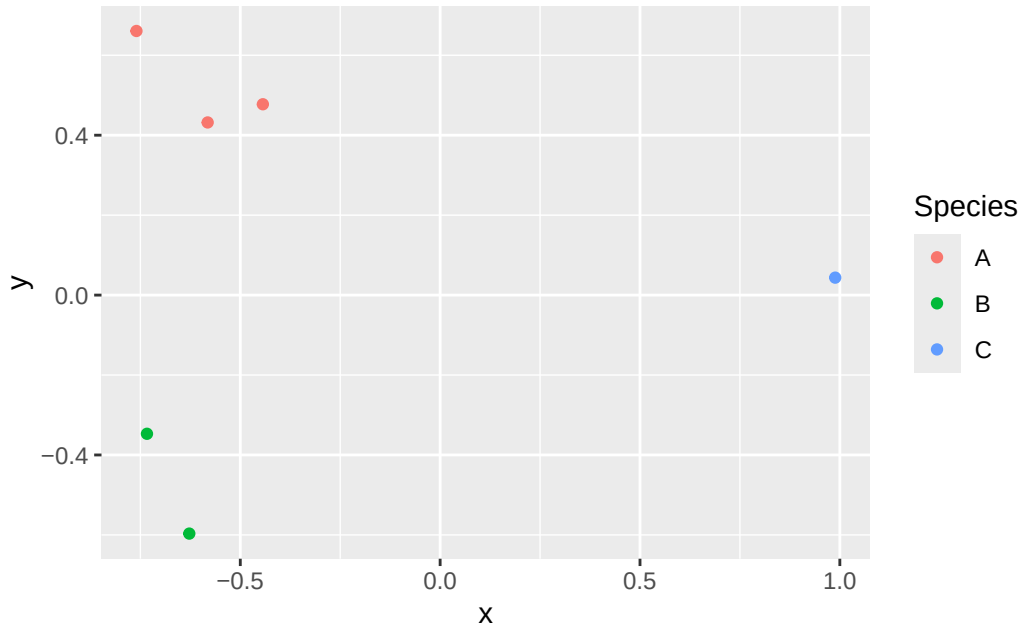
```
mock_pcoa$li
```

```
          A1            A2
A -0.6454972  5.000000e-01
B -0.6454972 -5.000000e-01
C  1.2909944  2.355139e-16
```

Plot individuals in the trait space, with a small variability.

```
# Create coordinates
tibble(
  Species = rep(names(mock_3sp_abd), times = mock_3sp_abd),
  x = rep(mock_pcoa$li[, 1], times = mock_3sp_abd),
  y = rep(mock_pcoa$li[, 2], times = mock_3sp_abd)
) ->
  mock_3sp_ind
# Choose variability
```

```r
variability <- 0.1
# Map
mock_3sp_ind %>%
  # Add variability
  mutate(
    x = x + rnorm(n(), mean = 0, sd = variability),
    y = y + rnorm(n(), mean = 0, sd = variability)
  ) %>%
  ggplot(aes(x = x, y = y, color = Species)) +
  geom_point() +
  coord_fixed()
```



Diversity of the 6 individuals. First, without variability.

```r
# Choose variability
variability <- 0
# Create the point set
mock_3sp_ind %>%
  # Add variability
  mutate(
    x = x + rnorm(n(), mean = 0, sd = variability),
    y = y + rnorm(n(), mean = 0, sd = variability)
  )  %>%
```

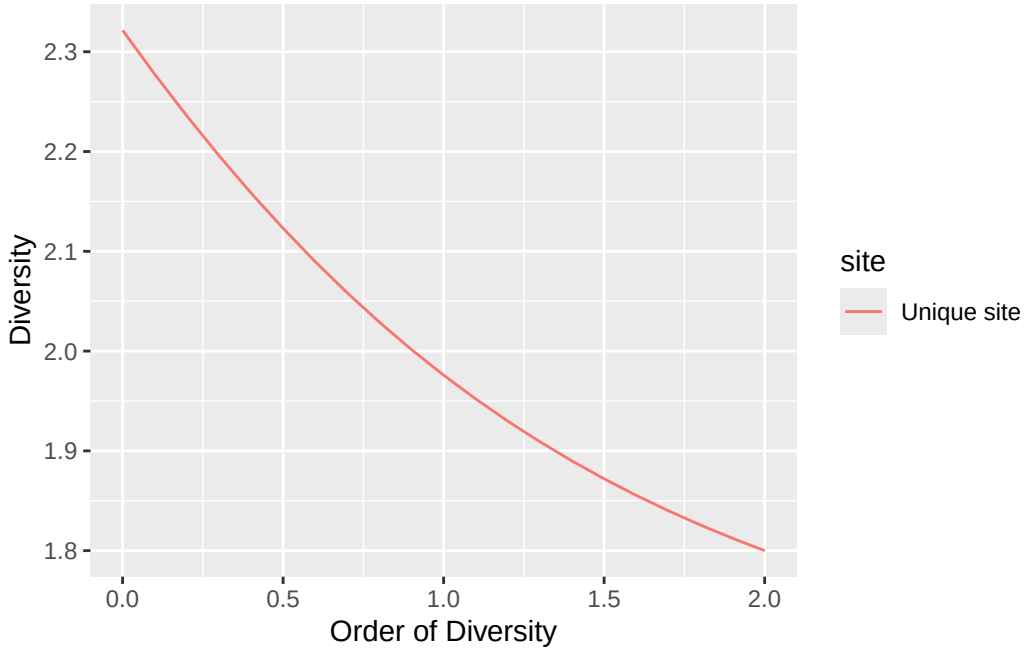
```
  # Distance matrix
  dist() %>%
  as.matrix() ->
  mock_6ind_dist
```

```
Warning in dist(.): NAs introduits lors de la conversion automatique
```

```
# Delete the row and column name of the matrix (they are not species names)
rownames(mock_6ind_dist) <- colnames(mock_6ind_dist) <- NULL
# Total abundance
mock_6ind_n <- nrow(mock_6ind_dist)
# Similarity
mock_6ind_Z <- 1 - mock_6ind_dist / max(mock_6ind_dist)
# Abundances of individuals are 1, or any identical value
mock_6ind_abd <- rep(1, mock_6ind_n)
# Diversity
profile_similarity(
  mock_6ind_abd,
  similarities = mock_6ind_Z,
  estimator = "naive"
) %>%
  autoplot()
```

```
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
```

```
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
! Sample coverage is 0, most estimators will return `NaN`.
```



Change the variability (0.1 first) and run the code again.

14 Diversity partitioning

14.1 Mock data

Three habitats: A, B, C.

```
matrix(
  c(
    0, 0, 0, 0, 0, 3, 3, 3, 3, 3,
    3, 3, 3, 3, 3, 3, 0, 3, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 3, 3, 3
  ),
  nrow = 3,
  byrow = TRUE
) %>%
```

```
  as_abundances() %>%
  mutate(
    site = paste("Habitat", LETTERS[1:3]),
    weight = 1 # weight = arbitraire, useful for example if you trust a dataset more than otl
  ) ->
  mock_island_abd
print(mock_island_abd)
```

```
# A tibble: 3 x 12
  site      weight  sp_1  sp_2  sp_3  sp_4  sp_5  sp_6  sp_7  sp_8  sp_9 sp_10
  <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Habitat A      1     0     0     0     0     0     3     3     3     3     3
2 Habitat B      1     3     3     3     3     3     3     0     3     0     0
3 Habitat C      1     0     0     0     0     0     0     0     3     3     3
```

Richness.

```
div_richness(mock_island_abd)
```

```
# A tibble: 3 x 5
  site      weight estimator   order diversity
  <chr>      <dbl> <chr>       <dbl>     <int>
1 Habitat A      1 Jackknife 0     0         5
2 Habitat B      1 Jackknife 0     0         7
3 Habitat C      1 Jackknife 0     0         3
```

```
# so here alpha = mean of 7, 5 & 3 = 5
div_richness(mock_island_abd, gamma = TRUE)
```

```
# A tibble: 1 x 4
  site          estimator   order diversity
  <chr>         <chr>       <dbl>     <dbl>
1 Metacommunity Jackknife 2     0        10
```

Multiplicative partitioning: 10 effective species (γ) = 5 effective species per habitat (α) times 2 habitats (β).

```
div_part(mock_island_abd, q = 0)
```

```
# A tibble: 6 x 6
  site          scale     estimator order diversity weight
  <chr>         <chr>     <chr>     <dbl>     <dbl>  <dbl>
1 Metacommunity gamma     "UnveilJ"     0        10      3
2 Metacommunity beta      ""            0         2     NA
3 Metacommunity alpha     ""            0         5     NA
4 Habitat A     community "UnveilJ"     0         5      1
5 Habitat B     community "UnveilJ"     0         7      1
6 Habitat C     community "UnveilJ"     0         3      1
```

Additive partitioning: 10 effective species (γ) = 5 effective species per habitat (α) + 5 more species (β).

= in average, each community has alpha species. It is not the single diversity of 1 habitat. It's a mean of several habitats. It's the average of **entropy**, not diversity. Here it is the same of calculating the mean of the diversity because we are computing the basic richness : deformed exponential = +1. On fait +1 et -1 donc ça revient au même. Si on s'intéresse à une diversité d'ordre supérieur le résultat de alpha sera différent. Calculate mean entropy and then take the exponential to get alpha diversity.

The number of habitats = number of habitats = = hill number.
It is proportionnal diversity : beta diversity : en moyenne, à quel point les plots sont différents de l'ensemble (gamma).

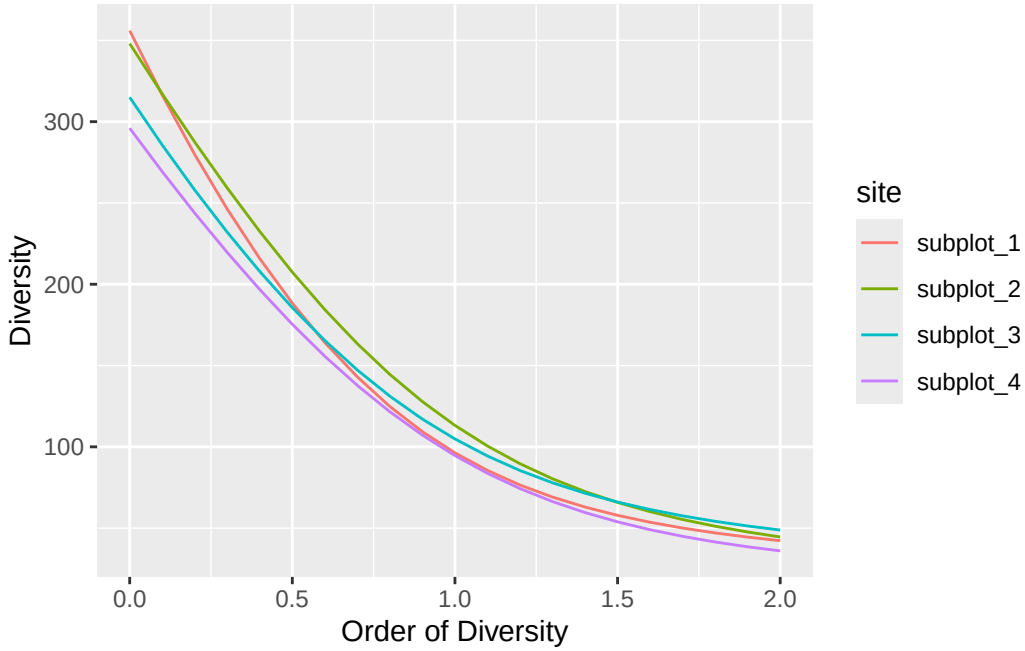
$$entropy : \gamma = \alpha + \beta$$

$$diversity : e^{\gamma} = e^{\alpha} * e^{\beta}$$

14.2 Paracou 6

The plot is made of 4 subplots.

```
# Diversity profiles of the 4 subplots
paracou_6_abd %>%
  profile_hill() %>%
  autoplot()
```



14.2.1 Partitioning

Function `div_part()` computes α, β and γ diversity.

```
# Ici on calcule l'entropie de Shannon
div_part(paracou_6_abd, q = 1)
```

```
# A tibble: 7 x 6
  site          scale     estimator order diversity weight
  <chr>         <chr>     <chr>     <dbl>     <dbl>  <dbl>
1 Metacommunity gamma     "UnveilJ"     1    111.     6.25
2 Metacommunity beta      ""            1      1.09  NA
3 Metacommunity alpha     ""            1    102.    NA
4 subplot_1     community "UnveilJ"     1     96.3    1.56
5 subplot_2     community "UnveilJ"     1    113.     1.56
6 subplot_3     community "UnveilJ"     1    105.     1.56
7 subplot_4     community "UnveilJ"     1     94.6    1.56
```

14.2.2 Step by step : what the computer does

```
# Alpha entropy
paracou_6_abd %>%
  ent_tsallis(q = 1) %>%
  # Mean entropy
  group_by(order) %>%
  summarise(ent_alpha = mean(entropy)) %>%
  # Diversity
  mutate(div_alpha = exp_q(ent_alpha, q = 1)) %>%
  print() ->
  paracou_6_alpha
```

```
# A tibble: 1 x 3
  order ent_alpha div_alpha
  <dbl>     <dbl>     <dbl>
1     1      4.62      102.
```

```
# Gamma entropy
paracou_6_abd %>%
  # Group the subplots
  metacommunity() %>%
  ent_tsallis(q = 1) %>%
  # Rename entropy for clarity
  rename(ent_gamma = entropy) %>%
  # Diversity
  mutate(div_gamma = exp_q(ent_gamma, q = 1)) %>%
  print() ->
  paracou_6_gamma
```

```
# A tibble: 1 x 6
  site          weight estimator order ent_gamma div_gamma
  <chr>          <dbl> <chr>     <dbl>     <dbl>     <dbl>
1 metacommunity   6.25 UnveilJ       1      4.71      111.
```

```
# Make a single tibble with alpha and gamma diversities
paracou_6_alpha %>%
  bind_cols(
    paracou_6_gamma %>%
      select(ent_gamma, div_gamma)
```

```
  ) %>%
  mutate(
    # beta entropy
    ent_beta = ent_gamma - ent_alpha,
    # beta diversity
    div_beta = div_gamma / div_alpha
  )
```

```
# A tibble: 1 x 7
  order ent_alpha div_alpha ent_gamma div_gamma ent_beta div_beta
  <dbl>     <dbl>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>
1     1      4.62      102.      4.71      111.   0.0826     1.09
```

14.2.3 Profiles

Not implemented yet in divent.

alpha diversity.

```
# Values of q used for computation: default values of profile_xxx()
orders <- seq(from = 0, to = 2, by = 0.1)
sapply(
  # Apply ent_tsallis to all orders
  orders,
  FUN = \(q) {
    # Get a number to obtain a matrix with sapply
    ent_tsallis(paracou_6_abd, q = q, as_numeric = TRUE)
  }
) %>%
  # Each row contains a subbplot, each column an order, values are entropy
  # Calculate mean entropy (weights are equal)
  colMeans() %>%
  # Diversity
  exp_q(q = orders) %>%
  # Make a tibble
  tibble(
    order = orders,
    div_alpha = .
  ) %>%
  print() ->
  paracou_6_alpha_profile
```

```
# A tibble: 21 x 2
   order div_alpha
   <dbl>     <dbl>
 1   0        329.
 2   0.1      297.
 3   0.2      267.
 4   0.3      239.
 5   0.4      213.
 6   0.5      189.
 7   0.6      167.
 8   0.7      148.
 9   0.8      130.
10   0.9      115.
# i 11 more rows
```

gamma diversity.

```
paracou_6_abd %>%
  profile_hill(gamma = TRUE) %>%
  # Rename diversity to recongnize gamma later
  rename(div_gamma = diversity) %>%
  print() ->
  paracou_6_gamma_profile
```

```
# A tibble: 21 x 3
   estimator order div_gamma
   <chr>     <dbl>     <dbl>
 1 UnveilJ     0        473
 2 UnveilJ     0.1      412.
 3 UnveilJ     0.2      357.
 4 UnveilJ     0.3      308.
 5 UnveilJ     0.4      265.
 6 UnveilJ     0.5      227.
 7 UnveilJ     0.6      195.
 8 UnveilJ     0.7      168.
 9 UnveilJ     0.8      145.
10 UnveilJ     0.9      126.
# i 11 more rows
```

Compute beta.

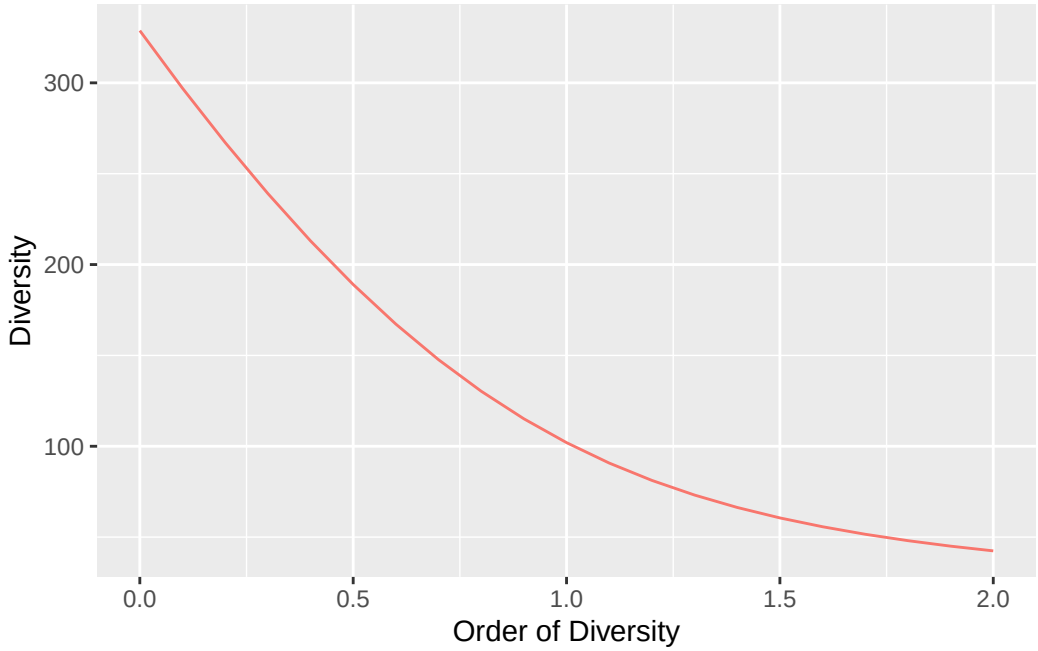
```r
paracou_6_gamma_profile %>%
  # Join the tibbles by identical order
  inner_join(paracou_6_alpha_profile) %>%
  # Calculate beta diversity
  mutate(div_beta = div_gamma / div_alpha) %>%
  print() ->
  paracou_6_profile
```

```
Joining with `by = join_by(order)`
```

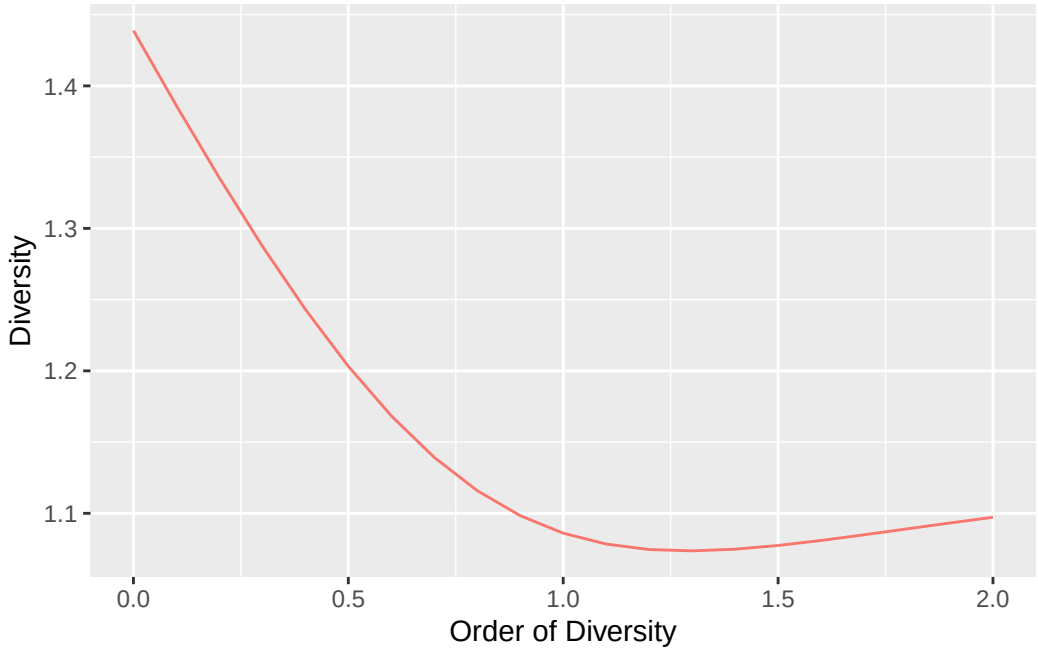
```
# A tibble: 21 x 5
   estimator order div_gamma div_alpha div_beta
   <chr>     <dbl>     <dbl>     <dbl>    <dbl>
 1 UnveilJ     0         473       329.     1.44
 2 UnveilJ     0.1       412.      297.     1.39
 3 UnveilJ     0.2       357.      267.     1.34
 4 UnveilJ     0.3       308.      239.     1.29
 5 UnveilJ     0.4       265.      213.     1.24
 6 UnveilJ     0.5       227.      189.     1.20
 7 UnveilJ     0.6       195.      167.     1.17
 8 UnveilJ     0.7       168.      148.     1.14
 9 UnveilJ     0.8       145.      130.     1.12
10 UnveilJ     0.9       126.      115.     1.10
# i 11 more rows
```

Plot.

```r
paracou_6_profile %>%
  # paracou_6_profile is an object of class profile that can be plotted
  # The column to plot must be named diversity
  rename(diversity = div_alpha) %>%
  autoplot() +
  theme(legend.position = "none")
```




```
paracou_6_profile %>%
  # rename div_beta to plot it
  rename(diversity = div_beta) %>%
  autoplot() +
  theme(legend.position = "none")
```



Berger, Wolfgang H., and Frances L. Parker. 1970. "Diversity of Planktonic Foraminifera in Deep-Sea Sediments." *Science* 168 (3937): 1345–47. https://doi.org/10.1126/science.168.3937.1345.

Chao, Anne, and Lou Jost. 2012. "Coverage-Based Rarefaction and Extrapolation: Standardizing Samples by Completeness Rather Than Size." *Ecology* 93 (12): 2533–47. https://doi.org/10.1890/11-1952.1.

Etienne, Rampal S. 2005. "A New Sampling Formula for Neutral Biodiversity." *Ecology Letters* 8 (3): 253–60. https://doi.org/10.1111/j.1461-0248.2004.00717.x.